Tunisian Republic
Ministry of Higher Education
Higher Institute Of Technological Studies Rades

---

*End-of-Studies Project Report*

**Higher Institute Of Technological Studies Rades**

*by*

**Mohamed Dhia SLEIMI**
**Mustapha BOUZIRI**

---

# MES System

Complete Project Report

---

***made at***

Mavision

# Acknowledgements

First and foremost, We would like to thank our academic supervisor, **Ms.** `Marwa CHAABANI`, for their expert guidance and unwavering commitment. Their in-depth knowledge, availability and unconditional support have been of paramount importance in the completion of this project.

We would also like to express our gratitude to our professional supervisor at **Mavision**, **Mr.** `Mourad YOUNSI`. Their contribution to this project, both on a technical and organisational level, has been invaluable. Their strategic vision, domain expertise and availability greatly facilitated our progress and allowed us to gain an enriching professional experience.

# Contents

# List of Figures

# List of Tables

# General Introduction

In an industrial context undergoing rapid digital transformation, manufacturing companies face the necessity of optimising their production processes, reducing downtime and ensuring complete traceability of workshop operations. Manufacturing Execution Systems, known by the acronym MES (*Manufacturing Execution System*), represent a direct response to these challenges by bridging the gap between decision-making layers (ERP) and the production floor.

It is within this context that the present end-of-studies project was carried out at **Mavision**. The project consists of the design and development of an integrated MES system, structured around two main components: a business back-end developed in AL language on the Microsoft Dynamics 365 Business Central platform, and a cross-platform mobile/web application developed with the Flutter framework. The system enables machine management, production order tracking, real-time production reporting, as well as user and role-based access management.

This report is structured into several chapters. The first chapter introduces the host organisation and presents the general framework of the project, highlighting the limitations of existing solutions and the relevance of the proposed solution. The second chapter focuses on requirements analysis and specification, as well as the technology choices made. The subsequent chapters trace the various development phases, exploring the conceptual aspects and illustrating the results obtained at the end of each sprint.

Through this approach, we aim to demonstrate how a well-designed digital solution can transform workshop production management by providing visibility, responsiveness and performance, serving both operators and management alike.

# Chapter 1

# Global Project Framework

## Introduction

This first chapter serves to contextualise our project. It begins with a brief introduction to the host organisation, then describes the objectives to be achieved, provides an in-depth analysis of the existing situation, and finally describes the methodology used to implement this work.

## 1.1 Presentation of the Host Company

To properly organise this section, we begin with some general details about Mavision before addressing the activities directly relevant to us.

### 1.1.1 Overview of Mavision

Founded in 2014 and headquartered in Manar II, Tunis, Mavision [1] is a software development company specialising in consulting, auditing and implementing ERP information systems. Mavision supports companies in their IT development projects and helps its clients better organise themselves around their information systems. In addition to its ERP offerings, Mavision develops and integrates reporting solutions to help its clients manage their business activities more effectively. In order to build lasting trust, Mavision places emphasis on its professionalism, commitment and the technical and functional expertise of its team.

**Figure 1.1.** Mavision Logo

### 1.1.2 Areas of Expertise

Mavision's activities are structured around three main areas:

**ERP Integration:** Consulting, implementation and integration of ERP solutions, with a strong focus on Microsoft Dynamics 365 Business Central, as well as project and platform management, architecture and custom business software design.

**Business Intelligence:** Design and deployment of BI solutions enabling clients to gain strategic visibility over their operations through dashboards, reporting tools and data analysis.

**Web & Mobile Technology:** Development and integration of mobile phone applications and web platforms that complement ERP systems, enabling clients to manage their business activities with greater agility and efficiency.

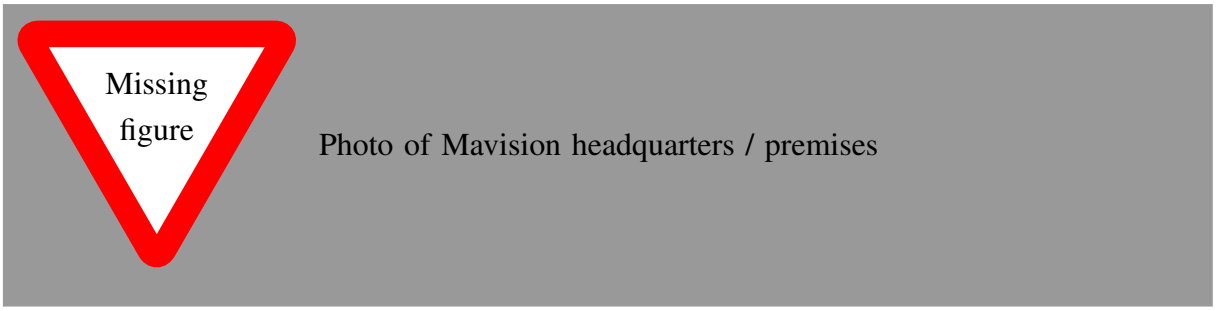


**Figure 1.2.** Mavision Headquarters

## 1.2 Project Framework

This project forms part of our end-of-studies internship, an essential step for obtaining the `bachelor's` degree at the `Higher Institute of Technological Studies`. This internship, lasting `four months`, took place from `February 2nd` to `May 30th` at Mavision. During this period, we were assigned to the development team.

This team is responsible for designing and maintaining software solutions that meet the company's business needs, leveraging modern technologies such as Microsoft Dynamics 365 Business Central and Flutter, and agile practices. Within this context, we worked on the development of the MES (*Manufacturing Execution System*) project, a mobile and web solution enabling intelligent management of machines and production operations on the workshop floor.

## 1.3 State of the Art

### 1.3.1 Subject Overview

With the rise of Industry 4.0 and the widespread adoption of ERP systems in manufacturing companies, the need to connect decision-making levels with floor operations has become critical. A MES (*Manufacturing Execution System*) is an information system that provides this link in real time: it supervises the execution of production orders, monitors machine status, records production declarations and provides operators and supervisors with a consolidated view of the workshop.

In this context, our project reflects a desire to provide Mavision and its industrial clients with a MES solution natively integrated with Microsoft Dynamics 365 Business Central. The primary objective is to allow operators to start and monitor their production operations from an intuitive mobile/web application, while ensuring data consistency with the ERP.

### 1.3.2 Problem Statement

The problem of optimising production floor operations arises with particular force in manufacturing companies that rely on an ERP system such as Microsoft Dynamics 365 Business Central. Indeed, while Business Central effectively handles production planning and order management

at the decision-making level, no unified, intelligent and dynamic solution exists to bridge the gap between the ERP and the shop floor in real time.

This gap leads to several consequences: inability to track the progress of production orders as they are being executed, manual entry of production declarations through paper routing sheets or spreadsheet files, lack of visibility for supervisors and managers on machine states and operation statuses, and insufficient traceability of actual shop-floor activity. Thus, a central question arises: how can Mavision provide its industrial clients with a solution that optimises the management of production operations, ensuring effective, accurate and real-time synchronisation between the shop floor and the ERP system?

### 1.3.3 Study of the Existing Situation

Following an in-depth analysis, it is clear that there is a genuine need within manufacturing companies for a centralised solution to efficiently manage production operations directly from the workshop floor.

Currently, production tracking is often carried out manually, via paper route sheets or spreadsheet files, leading to data entry errors, delays in information reporting and the absence of reliable traceability.

Supervisors and production managers have no real-time visibility into the progress of production orders, nor the current status of machines (running, paused, out of order). There is often no tool to automatically correlate production declarations with orders planned in the ERP.

### 1.3.4 Critical Analysis of the Existing Situation

In the context of this study, we briefly analysed the MES solution proposed by Delmia Apriso, one of the leading enterprise-grade Manufacturing Execution Systems on the market, whose interface is illustrated in the figure below.

Delmia Apriso is a comprehensive cloud-based MES platform developed by Dassault Systemes, designed for large-scale industrial enterprises. It covers the full spectrum of manufacturing operations including production execution, quality management, labour tracking and supply chain integration. While technically mature and feature-rich, it targets large multinational manufacturers with complex and highly customised deployments, resulting in significant licensing, implementation and maintenance costs that place it out of reach for small and medium-sized manufacturers.


**Figure 1.3.** Logo of the Delmia Apriso solution

From Mavision's perspective as a software provider, the goal is not to use an MES solution internally, but to deliver one to its industrial clients. The solution must therefore be cost-effective to sell, straightforward to deploy and maintain at client sites, and natively integrated into Microsoft Dynamics 365 Business Central, which is already Mavision's core ERP offering. A heavyweight platform such as Delmia Apriso, which requires a separate infrastructure and lengthy implementation cycles, is incompatible with this commercial model. Table 1.1 summarises the key attributes of Delmia Apriso against the specific needs of the project.

**Table 1.1.** Comparison table with Delmia Apriso

| **Features** | **Project needs** | **Delmia Apriso** |
|---|---|---|
| Real-time production order tracking | Yes | Yes |
| Machine status management | Yes | Yes |
| Production declaration from mobile | Yes | Partial |
| Native Business Central integration | Yes | No |
| Cross-platform mobile interface | Yes | Partial |
| Role management (Admin / Supervisor / Operator) | Yes | Yes |
| Secure token-based authentication | Yes | Yes |
| Adaptability to internal needs | High (custom solution) | Low (proprietary platform) |
| Ease of deployment at client sites | High | Low |
| Compatible with SME commercial model | Yes | No |

### 1.3.5 Proposed Solution

The diagnosis of the current situation has highlighted several shortcomings and limitations, as detailed in the previous section. To address these, we propose to design and deploy an innovative MES solution natively integrated with Microsoft Dynamics 365 Business Central.

This solution consists of two complementary parts:

- **An AL back-end (Business Central)**: developed in AL language, it exposes OData V4 endpoints enabling user management, token-based authentication, machine and operation tracking, as well as production declaration.
- **A Flutter application (front-end)**: a cross-platform mobile and web application allowing operators and supervisors to view production orders assigned to their machine, start, pause or complete an operation, declare produced quantities, and visualise progress in real time.

The objective is to optimise every phase of production management by establishing a reliable, traceable and ergonomic digital framework, serving both operators and management.

### 1.3.6 Objectives

Our MES solution aims to:

- Provide real-time tracking of machine status and production operations directly from the Business Central ERP.
- Allow operators to start, pause and complete operations from an intuitive mobile interface.
- Record production declarations (produced quantities, scrap) and synchronise them with production orders in Business Central.
- Manage users by roles (Administrator, Supervisor, Operator) with secure token-based authentication.
- Offer supervisors a consolidated view of production progress across their work centres.
- Reduce manual data entry and the risk of errors associated with paper-based or non-integrated processes.
- Provide an extensible architecture, ready to accommodate new features (statistics, alerts, QR/label integrations).

### 1.3.7 Adopted Methodology

The development method corresponds to an ordered and structured sequence of project phases, for efficient and optimal management.

### Selection of the Agile Methodology

Agile is an iterative and collaborative working method that takes into account the initial needs of the client and any changes that may arise. Its central ingredient is customer-centred iterative development, which allows a team to gather client feedback on a regular basis and make the necessary adjustments at the right time.

Thus, the client has a better overview of work progress, leading to regular sessions that guarantee the delivery of a functional application at all times throughout the improvement cycles. Consequently, the underlying key idea is to work faster in order to satisfy a client.

### Adoption of the Scrum Framework

We adopted the Scrum framework, an agile approach based on short sprints and regular ceremonies (Sprint Planning, Daily Stand-up, Sprint Review, Sprint Retrospective). This choice allowed us to deliver functional increments at the end of each sprint, while maintaining smooth communication with the professional supervisor at Mavision and our academic supervisor at the university.

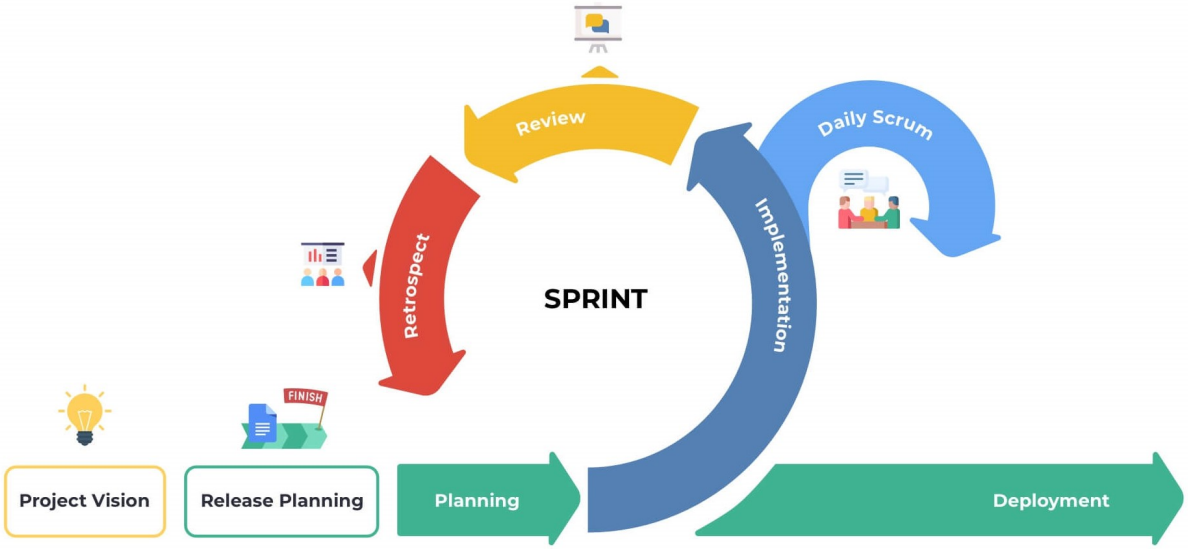


**Figure 1.4.** Adopted Scrum development cycle

## Conclusion

In this chapter, we presented Mavision, the company where the project is being carried out. We then defined the subject context, as well as the objectives of our work and the chosen methodology. In order to ensure the report progresses logically, we will examine the existing solutions used in the company. We will then explain the functional and non-functional requirements of our solution. This will allow us to evaluate existing solutions and identify the weaknesses and strengths for which we should make improvements to our solution. Finally, we will formulate the specific requirements for our solution.

# Chapter 2

# Requirements Analysis and Specification

## Introduction

Requirements analysis and specification is a crucial phase in the information system development process. It consists, first and foremost, of clarifying the functional and non-functional requirements of the system. This step then allows a complete understanding of the subject in order to position oneself clearly, precisely and without ambiguity.

## 2.1 Requirements Analysis

This step allows us to understand the overall context of our project, that is to say to identify the main features and determine the actors who will interact with the system.

### 2.1.1 Identification of Actors

Actors are entities external to the system (people, hardware, systems) that are likely to exchange information with it. The following is a list of the actors who will interact with our system:

**Table 2.1.** List of MES application users

| Actor | Description |
|---|---|
| Administrator | This is the person responsible for the MES platform. They manage all users (supervisors, operators), passwords and account statuses. They have access to all application features as well as the administration pages in Business Central. |
| Supervisor | This is an intermediate user. They can monitor the progress of production operations on their work centre, view machine status and track the progress of production orders. |
| Operator | This is the end user of the MES application. Assigned to a work centre (*Work Center*), they can view the production orders assigned to their machine, start, pause or resume an operation, and declare produced quantities. |

### 2.1.2 Identification of Requirements

At this stage, we will theoretically define the specific requirements of our application, clearly distinguishing functional requirements from non-functional requirements.

#### Functional Requirements

Each actor of the platform has specific expectations referred to as functional requirements. To define the functional scope, it is necessary that the conceptual solution meets these functional requirements.

- **Authentication**: This feature allows any user to log in to the platform using their Auth ID and password. Upon first login, the user is prompted to change their temporary password.

A secure session token (GUID, 12 h validity) is issued upon each successful login.

- **User Management**: This feature allows the administrator to:
  - View the complete list of MES users.
  - Create a new operator or supervisor account by linking it to a Business Central employee and assigning them a work centre.
  - Generate and assign a temporary password.
  - Activate or deactivate an account (automatic revocation of active tokens).
- **Machine Management**: This feature allows users to:
  - View the list of machines in a work centre with their current status (Idle, Starting, OutOfOrder).
  - View the production order currently running on each machine.
- **Production Order Management**: This feature allows the operator to:
  - View production orders (Planned, Firm, Released) assigned to their machine.
  - Filter and sort orders by status or date.
  - View the details of an order (item, quantity, planned dates, operation description).
- **Production Operation Management**: This feature is the core of the system. It allows the operator to:
  - Start an operation (validation of prerequisites: released order, free machine, previous operation completed if applicable).
  - Pause or resume an ongoing operation.
  - Declare produced quantities (validation: quantity $> 0$, no excess of the ordered quantity).
  - View progress in real time (produced quantity, completion percentage, operation status).
- **Progress Dashboard**: Allows the operator and supervisor to view all active operations (Running / Paused) on a given machine, with their respective progress.
- **Password Change**: Allows any logged-in user to change their password by entering their old password and the new one.

#### Non-Functional Requirements

To define the non-functional scope, it is necessary that the conceptual solution meets these non-functional requirements:

- **Performance**: The platform performs efficiently in terms of handling routine operations (login, order consultation, production declaration). Machine status updates are refreshed every 5 seconds via a streaming mechanism.
- **Security**: Implementation of a token-based authentication system (signed GUID, 12 h expiry) and role management (Admin, Supervisor, Operator) to control access to features. Passwords are stored as SHA-256 hashes with a salt.
- **Maintainability**: The AL back-end code is organised in layers (Tables, Enums, Codeunits, API Pages) and the Flutter code follows a layered architecture (Data, Domain, Presentation) with the Provider pattern, facilitating updates and the addition of new features.
- **Ergonomics**: The user interface is adaptive (mobile, tablet, PC) thanks to responsive layouts dedicated to each form factor, minimising the learning curve for workshop operators.
- **Portability**: The Flutter application is compiled for Android, iOS, Web, Linux, macOS and Windows from a single codebase. The Business Central back-end is deployable both on-premises and in SaaS (cloud) mode.

## 2.2 Project Management with Scrum

### 2.2.1 Backlog Features

## Domain 1: Authentication & User Management

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-1.1** | As a User (Operator / Supervisor / Administrator), I need to log into the system with my credentials so that I can access features appropriate to my role. | • User can enter username and password.<br>• System validates credentials against the ERP.<br>• Session token is created upon successful authentication.<br>• User is redirected to the appropriate dashboard based on role. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-1.2** | As an Administrator, I need to assign access to each user so that users only access features appropriate to their role. | • Ability to assign a user as Operator or Supervisor or Administrator.<br>• Unauthorized actions are blocked with clear error messages. | High |
| **US-1.3** | As an Administrator, I need to search users. | • Ability to filter users based on their role.<br>• Ability to search users based on name or email. | High |
| **US-1.4** | As an Administrator, I need to see a list of all MES users with their details so that I can manage accounts. | • The user list displays each account's full name, email, role, and work centre.<br>• Clicking a user row opens a password management dialog for that account.<br>• The list refreshes automatically after a new user is created. | High |
| **US-1.5** | As an Administrator, I need to generate and assign a temporary password to a user so that they can log in for the first time. | • Admin can generate a random secure password for any user.<br>• The generated password is sent to the ERP via `AdminSetPassword`.<br>• The target account is flagged *Need To Change Password* after the password is set.<br>• Success or failure is reported to the admin with a clear message. | High |
| **US-1.6** | As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current. | • Admin can select a new role for any user.<br>• Work-centre list updates based on the selected role.<br>• All active tokens for the target user are revoked on save.<br>• The user list refreshes after a successful change. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-1.7** | As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account. | • Admin can toggle the active status of any user.<br>• Deactivation immediately revokes all active tokens.<br>• An admin cannot deactivate their own account.<br>• The user row updates its visual status indicator immediately. | High |

## Domain 2: Machine Monitoring

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-2.1** | As an Operator or Supervisor, I need to see the list of machines in my work centre with their current status so that I can see which machines are currently active. | • Machine list is scoped to the authenticated user's work centre.<br>• Each card shows machine name, current status (*Idle*, *Starting*, *OutOfOrder*), and active production order number.<br>• The list refreshes automatically every 5 seconds. | High |
| **US-2.2** | As an Operator or Supervisor, I need to view the production orders assigned to a machine so that I know what work needs to be done. | • Only *Planned*, *Firm Planned*, or *Released* orders are shown.<br>• Each card displays order number, item description, planned quantity, planned start and end dates, and operation number.<br>• Orders can be searched and filtered by status, and sorted by planned start date. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-2.3** | As an Operator or Supervisor, I need to start a production operation so that the system records that the order has begun. | • Only *Released* routing lines may be started.<br>• An operation cannot be started if it is already running.<br>• A machine cannot run two operations simultaneously.<br>• A new MES Operation Status record is created with status *Running*. | High |
| **US-2.4** | As a Supervisor or Operator, I need to monitor the current status of all active operations on a machine so that I can follow the progress of each order. | • The Operations Progress tab shows all operations whose latest status record is *Running* or *Paused*.<br>• Each card shows order number, operation number, current status, and last updated timestamp.<br>• Data refreshes automatically every 5 seconds. | High |
| **US-2.5** | As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders and active operation progress without leaving the machine context. | • Selecting a machine from the list opens a dedicated machine page.<br>• The page provides at least two tabs: *Orders* and *Progress*.<br>• Switching tabs preserves the state of each tab without reloading data.<br>• The active tab is visually highlighted in the navigation bar. | High |
| **US-2.6** | As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the most relevant work. | • Orders can be searched by order number, item description, or date.<br>• Orders can be filtered by status (*All*, *Planned*, *Firm Planned*, *Released*).<br>• Orders can be sorted by planned start date ascending or descending.<br>• On mobile, the status filter is accessible via a compact popup menu. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-2.7** | As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production. | • The History tab shows all operations with status *Finished* or *Cancelled*.<br>• Each card shows order number, status badge, product name, produced quantity, start time, and end time.<br>• The list supports free-text search and sort by start date.<br>• Tapping a card navigates to the full operation detail page. | High |

## Domain 3: Production Execution & Traceability

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-3.1** | As an Operator or Supervisor, I need to declare the quantity produced in a cycle so that the system records production progress. | • Declared quantity must be positive and not exceed remaining quantity.<br>• Each declaration creates a new progression cycle record.<br>• Cumulative quantity and progress percentage update immediately. | High |
| **US-3.2** | As an Operator or Supervisor, I need to pause and resume an active operation so that interruptions are tracked accurately. | • A Running operation can be paused; a Paused one can be resumed.<br>• A machine cannot resume if another operation is already running. | High |
| **US-3.3** | As a Supervisor, I need to finish or cancel an operation so that the machine is released and the order is closed. | • Finishing a complete order (100%) shows a green confirmation dialog.<br>• Cancelling an incomplete order shows a red warning dialog.<br>• Closing sets the execution end time and inserts an Idle machine status. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-3.4** | As an Operator or Supervisor, I need a dedicated operation detail page with live data in one place. | • Shows status, order number, product, required and produced quantities.<br>• Live data streams without a manual refresh.<br>• Adapts between mobile and desktop layouts. | High |
| **US-3.5** | As an Operator or Supervisor, I need to see all production cycles declared for an operation. | • Cycle table lists operator, quantity, cumulative total, and timestamp.<br>• Records ordered newest-first with pagination.<br>• Refreshes after each new declaration. | High |
| **US-3.6** | As an Operator or Supervisor, I need a visual chart of declarations over time to identify performance trends. | • Line chart plots cycle quantity vs. declaration time.<br>• Touch tooltip shows operator, quantity, cumulative total, timestamp.<br>• Auto-scrolls to most recent data point on load and refresh. | Medium |
| **US-3.7** | As an Operator or Supervisor, I need to see the Bill of Materials for an operation to know which components are required. | • Components linked to the current routing step are colour-coded by availability (Available / Low Stock / Missing).<br>• Shared components not linked to this step are shown in neutral style.<br>• BOM data refreshes after every consumption event. | High |
| **US-3.8** | As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked. | • Scrap can only be declared when the operation is Running.<br>• A reason code must be selected before submission.<br>• Each declaration records quantity, code, and operator. | High |

Continued on next page

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-3.9** | As a Supervisor, I need to declare production on behalf of an operator so that output is attributed to the correct person. | • Supervisor role sees an operator selector in the declare dialog.<br>• Selected operator's ID is written to the Declared By field.<br>• If no operator is selected, the supervisor's ID is used. | Medium |
| **US-3.10** | As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded against the BOM. | • Live camera feed reads DataMatrix barcodes.<br>• Internal format parsed locally; external resolved via API.<br>• Duplicate scan increments quantity on the existing row.<br>• Confirming scans inserts consumption records and refreshes the BOM. | High |
| **US-3.11** | As an Operator or Supervisor, I need to print a label for a production order so that produced parts are correctly identified. | • Label includes DataMatrix barcode encoding order, machine, item data.<br>• PDF generation supports landscape/portrait toggle.<br>• Print button available only when operation is Finished or progress $\geq$ 100%. | Medium |
| **US-3.12** | As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information. | • One PDF page per declared unit with a label counter.<br>• Same DataMatrix encoding as the production label.<br>• Launched automatically after a successful production declaration. | Medium |

## Domain 4: Administration & Monitoring

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-4.1** | As an Administrator, I need a performance dashboard for all machines so that I can monitor production health across the facility. | • Dashboard shows uptime %, operation count, total produced, and scrap per machine.<br>• Configurable time-range (hours back) filter.<br>• Responsive grid: 1 col mobile, 2 col tablet, 3 col desktop. | High |
| **US-4.2** | As an Administrator, I need a filterable, paginated activity log of all system actions so that I can audit operator and machine activity. | • Log entries include type, operator, machine, action, and timestamp.<br>• Filterable by action type and time range.<br>• Paginated at 15 rows per page. | High |
| **US-4.3** | As an Administrator, I need a persistent sidebar navigation so that I can move between admin sections without losing context. | • Sidebar has sections for Users & Roles, Machine Dashboard, Activity Logs, and Settings.<br>• All pages kept alive simultaneously via IndexedStack.<br>• Active section is visually highlighted. | High |
| **US-4.4** | As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language. | • Language selector on login screen and in profile page.<br>• All UI strings externalised; no hard-coded text.<br>• Arabic locale activates RTL layout automatically.<br>• Language preference persisted across sessions. | High |
| **US-4.5** | As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation. | • Tutorials shown exactly once per install per screen.<br>• Coach marks cover: machine list, machine detail tabs, operation detail, and admin dashboard.<br>• Localised skip button text. | Medium |

### Remaining

| ID | User Story | Acceptance Criteria | Priority |
|---|---|---|---|
| **US-x.1** | As a Supervisor, I need to create a new production order so that I can respond to urgent production needs. | • Form captures: Machine, ProductNo, Planned Quantity, Start Time<br>• Validates product exists in ERP<br>• Validates machine can produce the product<br>• Order is saved to ERP database<br>• Order appears in machine's "Orders to be done" within a short time frame<br>• Audit trail captures order creation | Medium |
| **US-x.2** | As an Operator, I need to be able to declare production on another machine of the same type so that I can continue working when my terminal is broken. | • Operator can access machines of same type as their assigned machines<br>• System validates operator is authorized for that machine type<br>• Declaration process identical to US-4.1<br>• Audit trail shows which machine declaration was made from | Low |

## 2.3 Sprint Plan

The project was organised into four two-week sprints, each delivering a functional increment verified by a sprint review with the professional supervisor.

**Table 2.7.** MES project sprint plan

| Sprint | Domain | Scope |
|---|---|---|
| Sprint 1 | Authentication & User Management | Secure login, session token management, role-based access control, administrator user creation, password generation and reset, role and work-centre change, account activation/deactivation. |

Continued on next page

| Sprint | Domain | Scope |
|---|---|---|
| Sprint 2 | Machine Monitoring & Production Order Management | Real-time machine status tracking, production order list per machine, operation start, tabbed machine view with Orders / Progress / History tabs, order filtering and sorting, completed operation history. |
| Sprint 3 | Production Execution & Traceability | Production cycle declarations, pause/resume/finish/cancel operations, operation detail page with live data, production chart, Bill of Materials visibility, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing. |
| Sprint 4 | Administration, System Configuration | Administrator dashboard and activity log, admin navigation shell, multi-language support (EN/FR/AR) and RTL layout, in-app onboarding tutorials. |

Continued on next page

## 2.4 Global Application Architecture

### 2.4.1 Physical Architecture

The physical architecture of the MES system rests on two main nodes:

- **Business Central Server (on-premises / SaaS)**: hosts the AL extension that exposes OData V4 endpoints and the business REST APIs. In on-premises mode, the server runs on a BC210 instance accessible on port 7048. In SaaS mode, the base URLs follow the schema `api.businesscentral.dynamics.com/v2.0/<tenantId>/<environment>/ODataV4/`.
- **Flutter Client (mobile / web / desktop)**: application compiled for Android, iOS, Web or desktop, communicating with the BC server via HTTP POST requests to OData V4 endpoints and custom APIs.

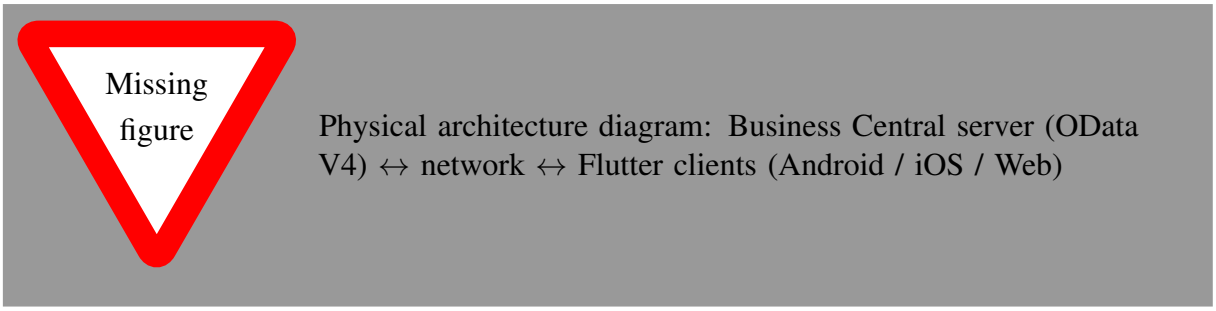


**Figure 2.1.** MES System Physical Architecture

### 2.4.2 Software Architecture

The software architecture of the system is organised around two distinct parts.

**AL Back-end (Business Central)**

The back-end is structured in six layers:

- **Tables (1-Tables)**: define the persistent data model — MES User, MES Auth Token, MES Machine Status, MES Operation Status, MES Operation Progression.
- **Enums (2-Enum)**: define the enumerated types — MES User Role (Operator, Supervisor, Admin), MES Machine Status (Idle, Starting, OutOfOrder), MES Operation Status (Running, Paused, Finished).
- **Codeunits (3-CodeUnit)**: contain the business logic — MES Auth Mgt (authentication, tokens), MES Password Mgt (SHA-256 hashing), MES Machine Actions (machine and operation management), MES Unbound Actions (OData V4 facade).
- **API Pages (4-API)**: expose data for reading/writing via the BC REST API — employees, MES users, work centres.
- **Permission Sets (5-PermissionSet)**: define access rights to AL objects for BC users.
- **Pages (6-Pages)**: administration and debugging interfaces in the BC client (lists, cards, API debug page).

**Flutter Front-end**

The Flutter front-end follows a three-layer architecture:

- **Data**: contains data models (models) and HTTP services (services) that communicate with BC endpoints via the `http` library.
- **Domain**: contains the providers (Provider pattern) that expose the application state to widgets — AuthProvider, MesUserProvider, MesMachinesProvider, MachineordersProvider, etc.

- **Presentation**: contains the pages and widgets of the user interface, organised by functional domain (auth, admin, machine) with dedicated responsive layouts (mobile, tablet, PC).

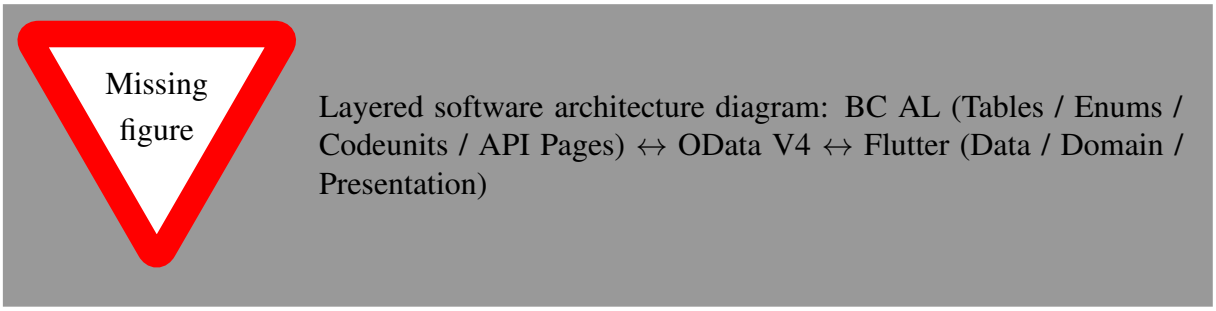


**Figure 2.2.** MES System Software Architecture

## 2.5 Work Environment

### 2.5.1 Software Environment

Table 2.8 summarises the main tools and technologies used in this project.

**Table 2.8.** MES project software environment

| Logo | Tool / Technology | Role in the project |
|---|---|---|
| | Microsoft Dynamics 365 Business Central | ERP platform hosting the AL back-end and business data |
| | AL Language (Application Language) | Development of the BC extension (tables, codeunits, API pages) |
| | Flutter (Dart) | Development of the cross-platform mobile/web application |
| | Visual Studio Code | Primary development environment (AL and Flutter) |
| | Git / GitHub | Version control and collaboration |
| | Postman | Testing and validation of OData V4 endpoints |

### 2.5.2 Development Environment

[DEVELOPMENT_ENVIRONMENT: description of the workstations used (OS, RAM, CPU), tool versions (Flutter SDK, Dart SDK, BC version, VS Code AL Language and Flutter extensions), and any specific configuration required (docker, BC on-prem instance, etc.). to be done later]

## Conclusion

In this chapter, we analysed and specified the functional and non-functional requirements of the MES system, identified the actors and their interactions, and presented the overall architecture of the solution. We also described the work environment and the adopted Scrum methodology. The following chapters detail the work carried out sprint by sprint, presenting the conceptual aspects and the results obtained at the end of each iteration.

# Chapter 3

# MES Sprint 1 Repport

## Contents

## Introduction

This sprint focuses on delivering a complete authentication and user management module for the *MES* application. The primary goal is to develop a secure authentication system covering *login*, *logout*, and password management, alongside role-based page access control to ensure each user can only reach features appropriate to their role.

In addition, the Administrator is given the ability to add new MES users, generate secure passwords on their behalf, change a user's role and work-centre assignment, and activate or deactivate accounts. By the end of this sprint, a fully functional *login* screen, *change password* flow, *admin user management dashboard*, *role management dialog*, and *logout* functionality will all be in place.

## 3.1 Sprint Backlog

This sprint covers **Domain 1:** ***Authentication & User Management***.

**Table 3.1.** Story Backlog

| ID | User Story | Tasks |
|---|---|---|
| **US-1.1** | As an Administrator, I need to assign access to each user so that users only access features appropriate to their role. | *Business Central (AL)*<br>• Implemented the functions `AdminCreateUser()`, and `AdminSetActive()` in codeunit MES Unbound Actions.<br>• Exposed `AdminCreateUser()`, and `AdminSetActive()` through codeunit MES Web Service.<br>• Created API page MES User Create API.<br>*Flutter*<br>• Updated ApiService to consume the new endpoints.<br>• Implemented MesUserService to consume `fetchMesUsers()` and `createMesUser()` endpoints.<br>• Created provider MesUserProvider to manage its state.<br>• Created widget AddUserDialog.<br>• Implemented role-based navigation in _AuthGate. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| **US-1.2** | As a User (Operator / Supervisor / Administrator), I need to log into the system with my credentials so that I can access features appropriate to my role. | *Business Central (AL)*<br>• Created the tables MES USER and MES AuthToken.<br>• Created enum MES UserRole.<br>• Implemented functions `MakeSalt()`, `HashPassword()`, and `VerifyPassword()` in codeunit MES Password Mgt.<br>• Implemented functions `CreateUser()`, `SetPassword()`, `ValidateCredentials()`, `IssueNewToken()`, `ValidateToken()`, `TouchToken()`, `Logout()`, `ValidateChangePassword()`, `ValidateAdminToken()`, `SetActive()`, and `CleanupExpiredTokens()` in codeunit MESAuthMgt.<br>• Implemented functions `Login()`, `Logout()`, `Me()`, and `ChangePassword()` in codeunit MES Unbound Actions.<br>• Exposed `Login()`, `Logout()`, `Me()`, and `ChangePassword()` in codeunit MES Web Service.<br>*Flutter*<br>• Implemented ApiService to consume the new endpoints.<br>• Created provider AuthProvider to manage its state.<br>• Implemented authentication routing in _AuthGate.<br>• Created page LoginPage.<br>• Created AppConstants. |
| **US-1.3** | As an Administrator, I need to search users. | *Flutter*<br>• Implemented user search in AddUserPage.<br>• Implemented role filtering in AddUserPage.<br>• Combined role and search filters in AddUserPage. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| **US-1.4** | As an Administrator, I need to see a list of all MES users with their details so that I can manage accounts. | *Business Central (AL)*<br>• Created the function fetchAllMESUsers in codeunit MES Unbound Actions.<br>• Exposed fetchAllMESUsers through codeunit MES Unbound Actions.<br>*Flutter*<br>• Created model MesUser.<br>• Implemented `MesUserService` to consume the new end point.<br>• Implemented `MesUserProvider` to manage its state and its streams.<br>• Implemented user list display in AddUserPage.<br>• created the widgets GeneratePasswordDialog and EmployeeAvatar. |
| **US-1.5** | As an Administrator, I need to generate and assign a temporary password to a user so that they can log in for the first time. | *Business Central (AL)*<br>• Implemented function `AdminSetPassword()` in codeunit MES Unbound Actions.<br>• Exposed `AdminSetPassword()` through codeunit MES Web Service.<br>*Flutter*<br>• Created widget GeneratePasswordDialog.<br>• Updaed ApiService to consume `adminSetPassword()`.<br>• Updated AuthProvider to account for the new functions in the service. |
| **US-1.6** | As an Administrator, I need to change the role and work-centre assignment of an existing user so that access rights stay current. | *Business Central (AL)*<br>• Implemented function `AdminChangeUserRole()` in codeunit MES Unbound Actions.<br>• Exposed `AdminChangeUserRole()` through codeunit MES Web Service. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| | | *Flutter*<br>• Update MesUserService to consume `changeUserRole()`.<br>• Update MesUserProvider to account `changeUserRole()`.<br>• Created widget ChangeRoleDialog. |
| **US-1.7** | As an Administrator, I need to activate or deactivate a user account so that access can be revoked without deleting the account. | *Flutter*<br>• Updated ApiService to account for `toggleUserActiveStatus()`.<br>• Updated AuthProvider to account for `toggleUserActiveStatus()`.<br>• Integrated activate and deactivate actions in AddUserPage. |

## 3.2 Business Rules

**Table 3.2.** Business Rules

| ID: Business Rule | Description |
|---|---|
| **Authentication Rules** | |
| `BR-AUTH-001` Credential Validation | • Credentials must be validated against the ERP database.<br>• Generic validation failure to prevent user enumeration. |
| `BR-AUTH-002` Session Management | • Each session generates a unique token (*GUID*).<br>• Token lifetime: 12 hours.<br>• Token expires automatically after this period.<br>• Only one active token per device. |
| `BR-AUTH-003` Password Security | • Minimum length: 8 characters.<br>• Must contain uppercase, lowercase, number, and special character.<br>• Encryption using *PBKDF2-SHA256*.<br>• Unique salt per user (randomly generated). |
| **User Administration Rules** | |

Continued on next page

| ID: Business Rule | Description |
|---|---|
| BR-ADMIN-001 Role–Work-Centre Constraint | • An Operator or Supervisor must be assigned to at least one work centre.<br>• An Administrator must have no work-centre assignment.<br>• These constraints are enforced both in `AdminCreateUser()` and `AdminChangeUserRole()`. |
| BR-ADMIN-002 Role Change Revokes Active Tokens | • `AdminChangeUserRole()` calls `RevokeAllTokensForUser()` after updating the role and work-centre fields, forcing the affected user to re-authenticate and receive a new session reflecting their updated permissions.<br>• The same revocation logic applies when an account is deactivated via `AdminSetActive()`. |

## 3.3 Design

### 3.3.1 Use Case Diagram

The UML Use Case Diagram (Figure 3.1) shows the three primary actors (Operator, Supervisor, Administrator) and their interactions with the *MES* authentication and user management system. In addition to the core login, logout, change password, and user creation flows, the diagram includes the *Change Role / Work Centre* and *Activate / Deactivate Account* use cases available exclusively to the Administrator.

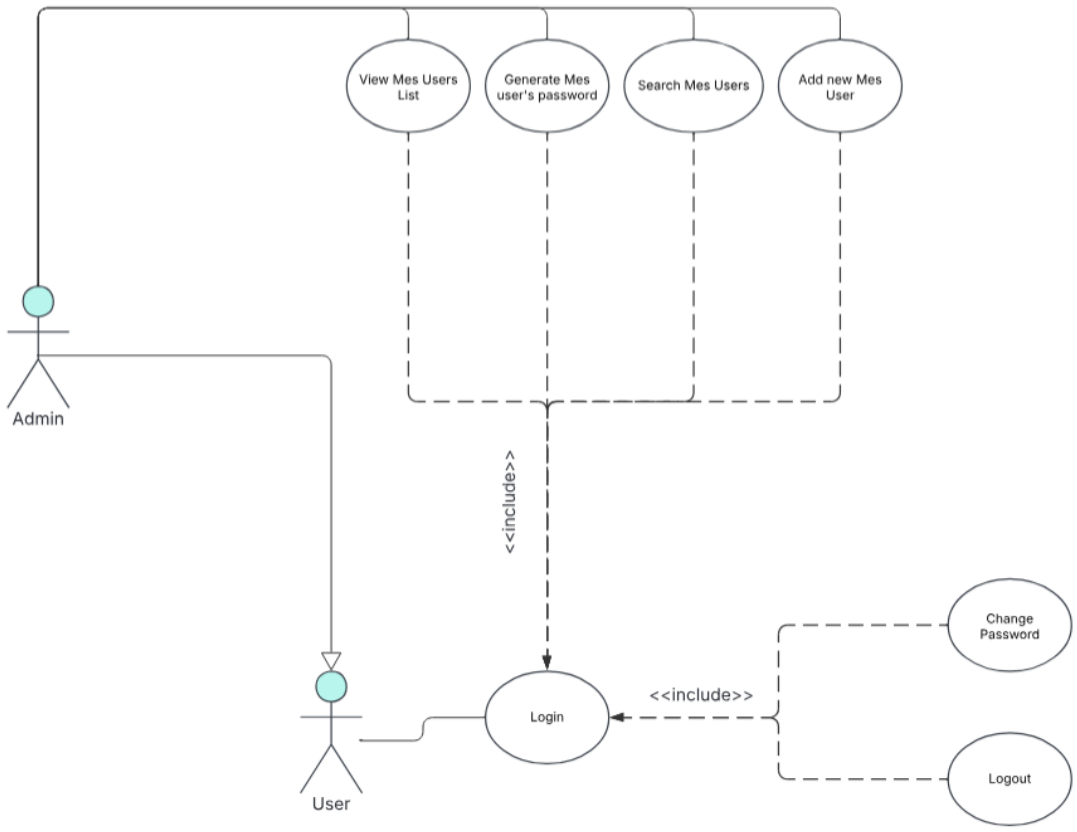


**Figure 3.1.** UML Use Case Diagram.

### 3.3.2 Textual Use Case Description

| | |
|---|---|
| **Use Case Name** | Login |
| **Primary Actors** | Operator, Supervisor, Administrator |
| **Preconditions** | • The user account exists in the ERP system.<br>• The user has an assigned role and department.<br>• The account must have a password (there is a possibility that there is an account without a password).<br>• The user account is active. |

Continued on next page

| | |
|---|---|
| **Postconditions** | • If the credentials are valid, the user is successfully authenticated.<br>• A session token is generated and stored securely.<br>• The user is redirected to the appropriate dashboard based on their role (Admin or MES interface).<br>• If the *need change password* flag is set to true, they are redirected to the *Change Password* page.<br>• If the credentials are invalid, access is denied and an error message is displayed. |
| **Main Scenario** | 1. The user launches the application.<br>2. The system displays the *login screen*.<br>3. The user enters *User ID* and *password*.<br>4. The system automatically detects the device ID.<br>5. The user clicks the *Login* button.<br>6. The system validates the credentials against the ERP database.<br>7. The system generates a secure session token.<br>8. The system checks the user role.<br>9. The system redirects the user to the appropriate dashboard (Admin or MES). |

**Table 3.3.** Textual Use Case Description — Login

### 3.3.3 Sequence Diagrams

The following high-level sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for each authentication flow. Each diagram shows the frontend-to-API-to-Business-Central interaction, including all success and error branches.

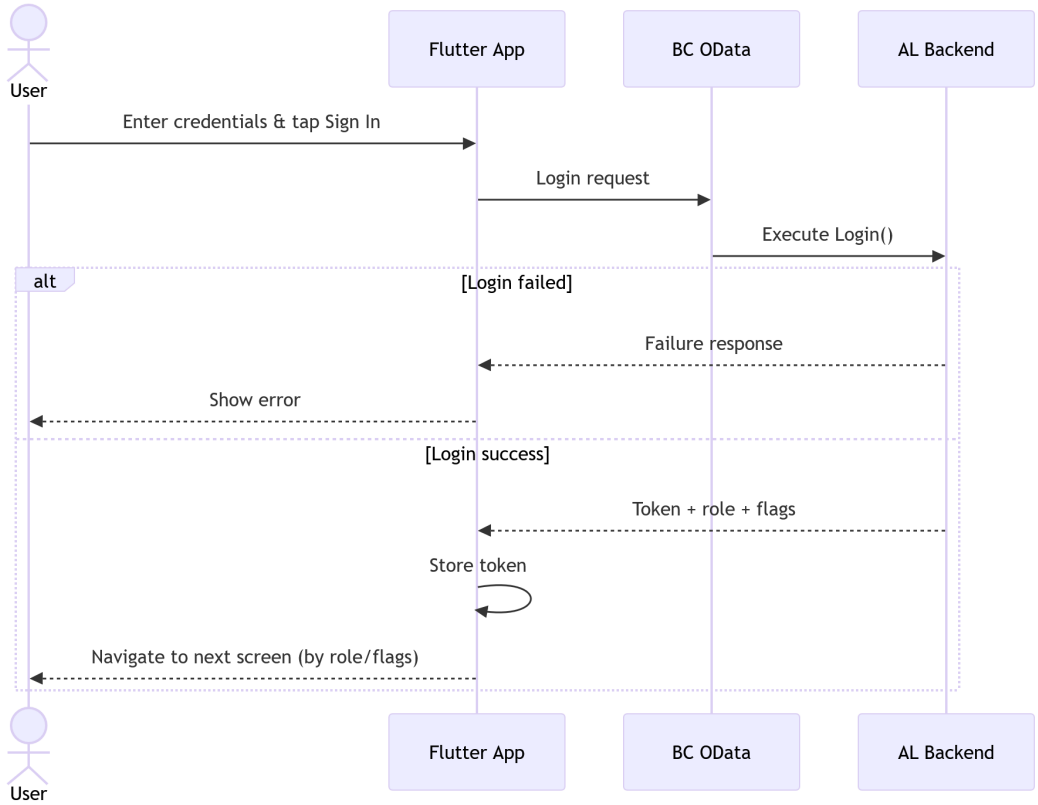


**Figure 3.2.** Login — Sequence Diagram.

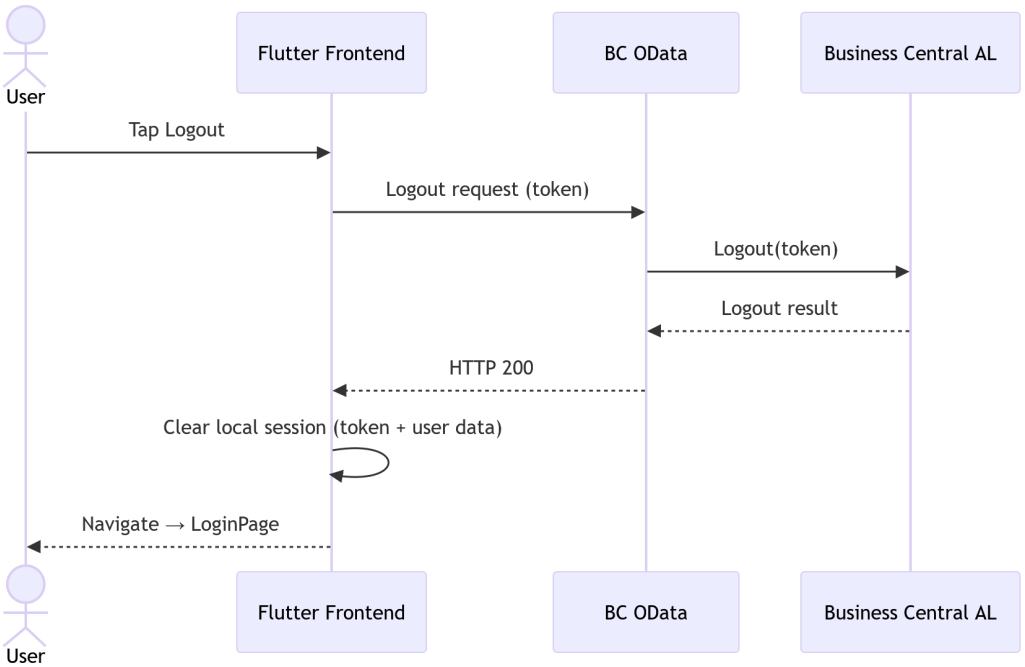


**Figure 3.3.** Logout — Sequence Diagram.

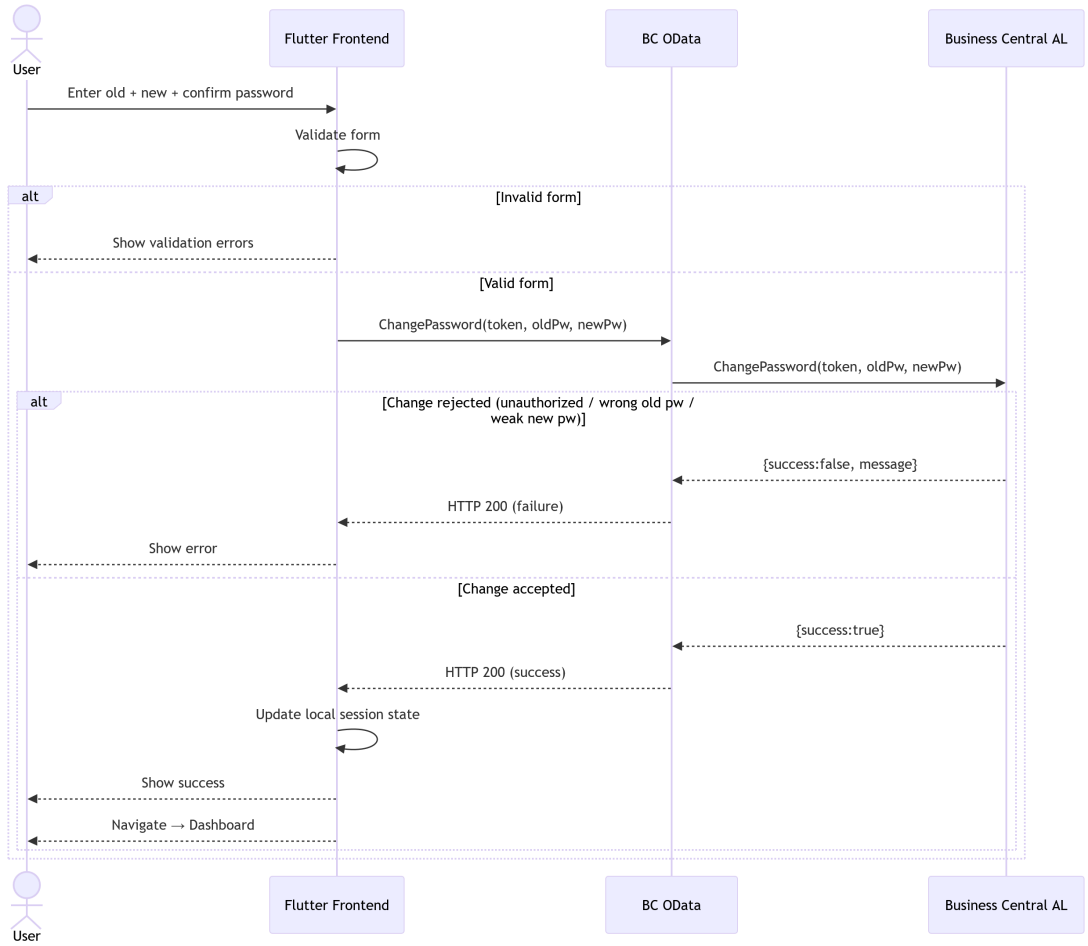


**Figure 3.4.** Change Password — Sequence Diagram.

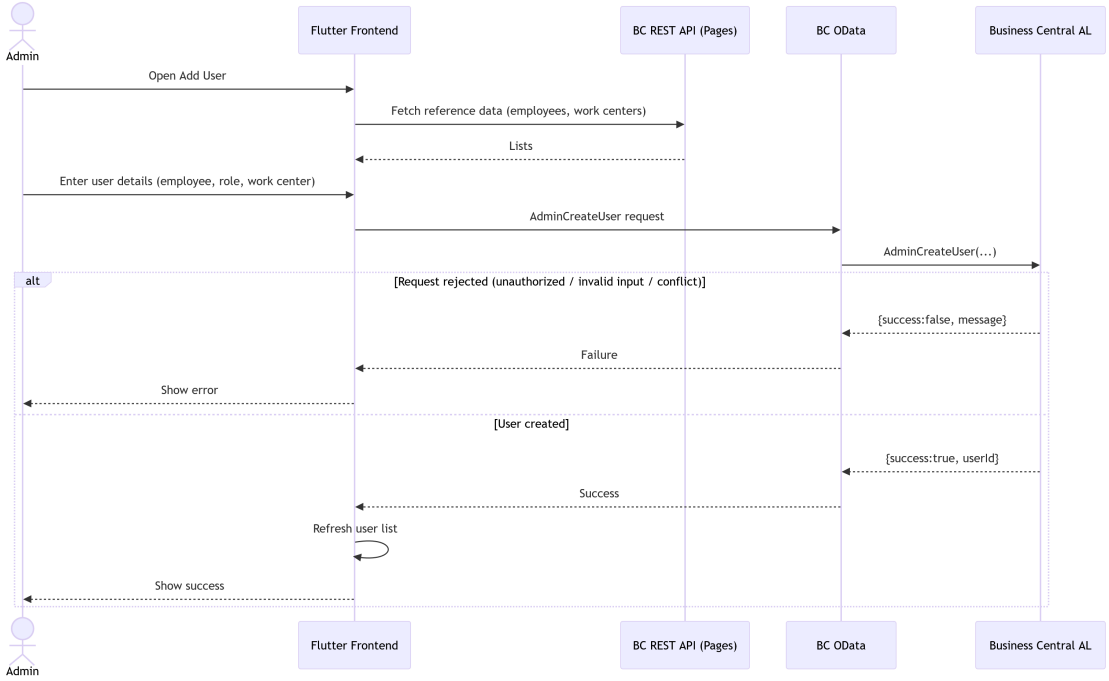


**Figure 3.5.** Admin — Create User — Sequence Diagram.

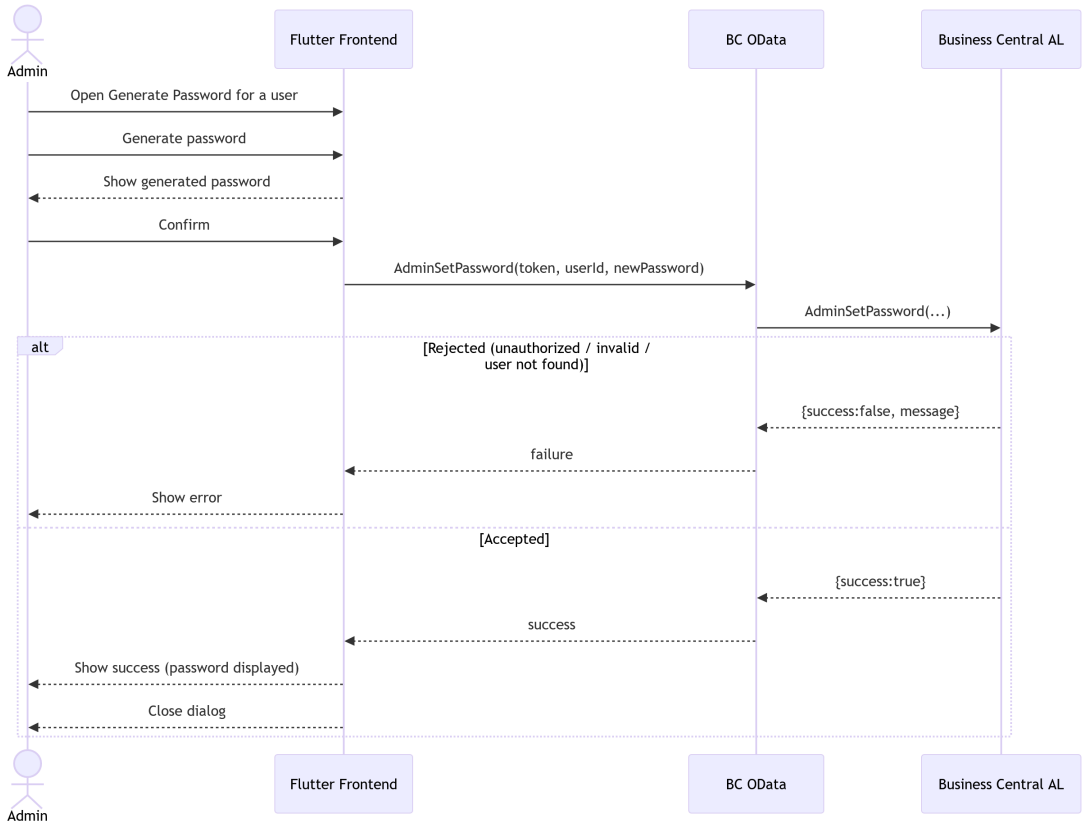


**Figure 3.6.** Admin — Set Password — Sequence Diagram.



**Figure 3.7.** Admin — Change Role / Work Centre — Sequence Diagram.

**Figure 3.8.** Admin — Toggle Account Active Status — Sequence Diagram.

### 3.3.4 Class Diagram

The UML Class Diagram of the *MES* authentication domain, shown in Figure 3.9, depicts the `MesUser`, `AuthToken`, and `PasswordPolicy` classes with their attributes and associations.

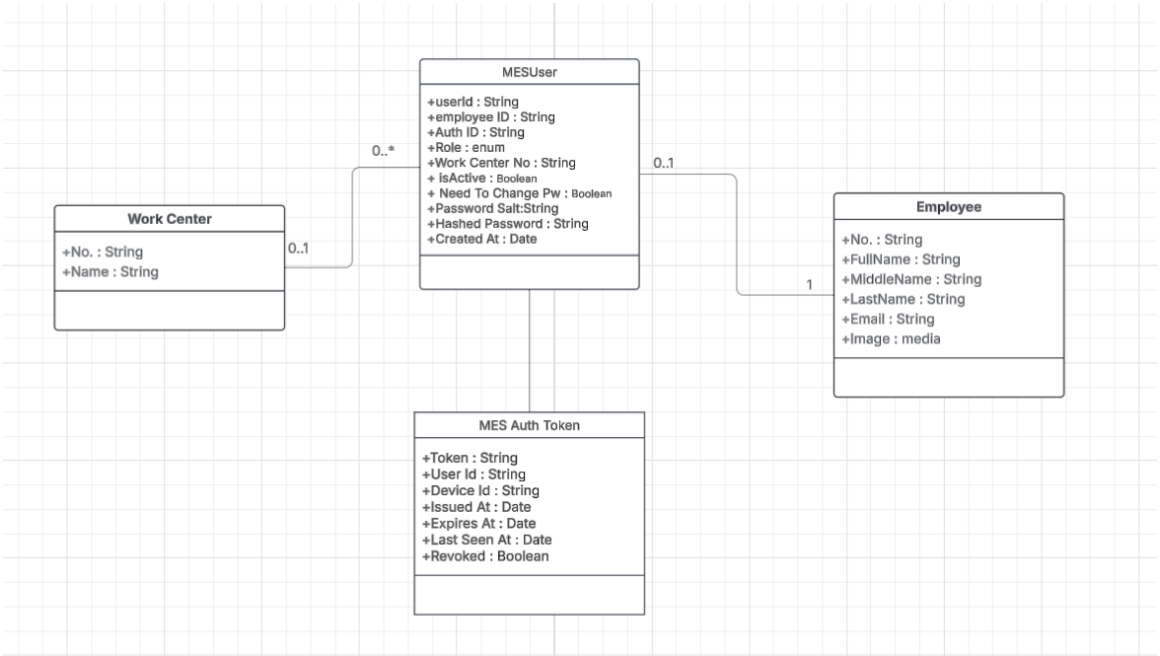


**Figure 3.9.** UML Class Diagram.

## 3.4 Implementation

### 3.4.1 Backend Implementation

#### Backend Structure Overview

The backend project structure is illustrated in Figure 3.10.

```
AL-backend/
          src/
                1-Tables/
                      NEW/
                          MES_AuthToken.al
                          MES_USER.al
                2-Enum/
                      NEW/
                          MES_UserRole.al
                3-CodeUnit/
                      NEW/
                          MESAuthMgt.al
                          MESPasswordMgt.al
                          MESSetup.al
                4-API/
                      NEW/
                          MESAuthActions.al
                          MESAuthApi.al
                5-PermissionSet/
                      NEW/
                          MESAuthApi.permissionset.al
                6-Pages/
                    NEW/
                        MESApiDebug.Page.al
                        MESUserCard.Page.al
                        MESUserList.Page.al
```

**Figure 3.10.** Backend project structure.

#### MES User Table Implementation

The `MES User` table is implemented to store user information, as described in Table 3.4.

| Name | Type | Description |
|---|---|---|
| User Id | Code[50] | Primary key. Login identifier used at the MES login prompt (e.g., `JDOE`, `OP001`). If blank on insert, a GUID-derived value is assigned automatically. |
| Employee ID | Code[50] | Foreign key to `Employee."No."`. Required for every MES account and enforced as unique (one Employee maps to one MES User). |

Continued on next page

| Name | Type | Description |
|---|---|---|
| Auth ID | Text[100] | Auto-generated external identifier in the form `AUTH-XXXXXXXX`. Used for display/external references (not used for login). Uniqueness enforced. |
| Role | Enum (`MES User Role`) | Authorization role for the MES application (e.g., Operator / Supervisor / Admin). Used to control permissions and access. |
| Work Center No. | Code[20] | Foreign key to `Work Center."No."`. Work center assignment: required for Operator and Supervisor; must be blank for Admin. |
| Is Active | Boolean | Account enable/disable flag. If false, authentication is blocked and existing tokens are revoked when deactivated. |
| Need To Change Pw | Boolean | If true, the user must change their password before using the MES application (set at creation and on forced password resets). |
| Password Salt | Text[64] | Random hex salt used in password hashing so identical passwords produce different hashes across users. |
| Hashed Password | Text[128] | Hashed password value (hex string). Passwords are never stored in plaintext; hashing uses the password plus salt. |
| Created At | DateTime | UTC timestamp set on insert only (audit trail/reporting). |

**Table 3.4.** MES User (Table 50101) — Field Dictionary

### Authentication Token Table

The `MES Auth Token` table stores session information as described in Table 3.5.

| Name | Type | Description |
|---|---|---|
| Token | Guid | Primary key. Raw GUID token presented by the client as a Bearer credential. Generated at issue time (never reused). Used for all direct token lookups (validation/logout). |
| User Id | Code[50] | Foreign key to `MES User."User Id"`. Identifies the MES user who owns this session/token. Enforces referential integrity via `TableRelation`. |

Continued on next page

| Name | Type | Description |
|---|---|---|
| Device Id | Text[100] | Optional identifier for the device that requested the token (e.g., `tablet-floor-02`, `scanner-01`). Stored for traceability/audit only; not used for validation. |
| Issued At | DateTime | UTC timestamp set when the token is created. Useful for audit trail and token age inspection. |
| Expires At | DateTime | UTC timestamp after which the token must be rejected even if not revoked. Set to `Issued At + TTL`. Token validation enforces `Expires At > CurrentDateTime()`. |
| Last Seen At | DateTime | Updated on every successful token validation. Supports session monitoring (activity/idle detection). Not used for expiry enforcement (use `Expires At`). |
| Revoked | Boolean | If true, token is explicitly invalidated and must be rejected even if not expired. Set on logout and during bulk revocation (e.g., password change, account deactivation). |

**Table 3.5.** MES Auth Token (Table 50106) — Field Dictionary

Key points of the token design include a unique *GUID* token, auto-expiry after 12 hours, last activity tracking, and soft revocation.

**Password Management and Encryption**

The password management codeunit ensures that plaintext is never persisted. A 50-character random *salt* is generated per user and the password is hashed using *PBKDF2-SHA256* with 1000 iterations before storage. Data classification attributes mark sensitive fields as *Customer Content* or *System Metadata*.

Key points include: passwords are never stored in plaintext; a unique salt per user (50 characters) is generated; *PBKDF2* hashing with 1000 iterations is applied; and data is classified as *Customer Content* or *System Metadata*.

**Authentication Logic — Login, Logout, and Change Password**

**Login**

The login validates the user credentials against the `MES User` table. If the credentials are correct and the account is active, a token will be generated and stored in the `MES Auth Token` table.

The system returns a *JSON* login response object which includes the generated session token, the authenticated user's role string, and a boolean *needChangePassword* flag consumed by the

Flutter frontend to decide the post-login navigation target.

**Logout**

On logout, the user's session authentication token will be expired. The token record matching the supplied session token is located in the `MES Auth Token` table and its expiry timestamp is set to the current datetime, immediately invalidating the session (soft revocation).

**Change Password**

Changing password requires a valid authentication token, a correct current password, and a new password that complies with security rules.

### 3.4.2 REST API and Web Services Implementation

To enable communication between Flutter and Microsoft Dynamics Business Central, a set of *RESTful APIs* were implemented using AL.

**API Pages for MES User Management**

The *API Page* definition for the `/mesUsers` endpoint exposes the `MES User` table as a read-only *OData* feed.

The *API Page* definition for the `/createMesUser` endpoint differs from the read endpoint by setting validation triggers. `DelayedInsert = true` buffers all field values until the complete request body has been received before committing the new `MES User` record to the database.

Key point: the difference between these two APIs is the *Delayed Insert* flag. When set to `true`, it delays insertion until all fields are received, which is required for `POST` operations. `GET` is not affected.

**OData Functions and Web Service Configuration**

Built MESUnboundActions codeunit (50125): `Login()`, `Logout()`, `Me()`, `ChangePassword()`. Exposed these functions in the MES Web Service codeunit (50126) exposed via ODataV4. Configured WebServices.xml to publish MES Web Service (codeunit 50126) as an ODataV4 web service under the service name `MESWebService`.

### 3.4.3 Frontend Implementation

The frontend of this system was developed using Flutter. The application is structured into four main layers: a ***UI Layer*** consisting of Screens and Widgets; a ***State Management Layer*** built on Providers; a ***Service Layer*** for API communication; and a ***Model Layer*** for Data Models.

The data flow follows a unidirectional pattern from the *UI Layer* through the *State Management Layer* (`Provider` pattern) and *Service Layer* to the Business Central REST API backend, as illustrated in Figure 3.11.

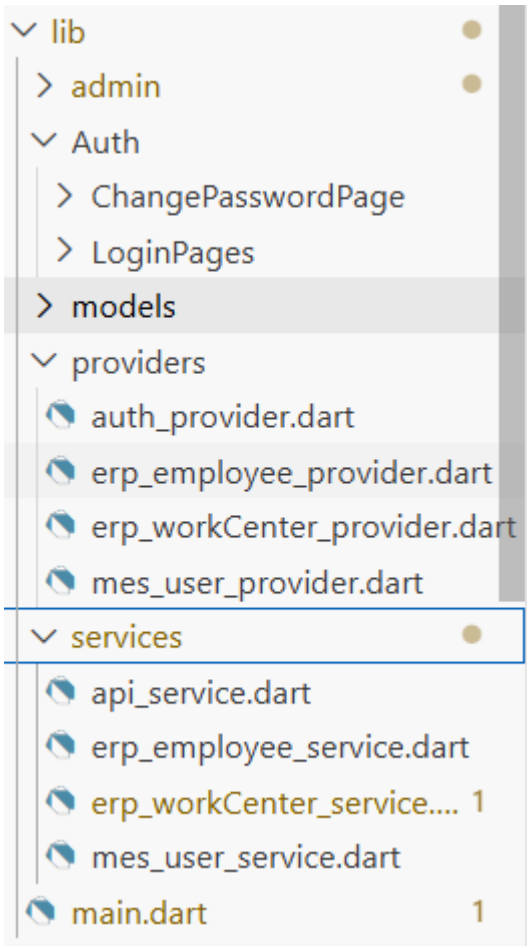


**Figure 3.11.** Flutter layered architecture.

### Service Layer (API Communication)

The *Service Layer* is responsible for handling all *HTTP* communication between Flutter and the Business Central backend.

Two services were implemented: the `api_service` manages authentication requests such as *login*, *logout*, and *password change*, while the `mes_user_service` handles MES user operations including fetching and creating new users.

`api_service.dart`

The class declaration and constructor show the base URL configuration and *HTTP* client initialisation used by all authentication endpoints. The *login* method sends user credentials and device ID to the `/login` endpoint and parses the returned session token, role, and *needChangePassword* flag. The *logout* method sends the active session token to the `/logout` endpoint to trigger server-side token revocation. The *change password* method submits the current password, new password, and session token to the `/changePassword` endpoint. Shared error-handling and response-parsing utilities are used by all methods in the service class to propagate structured exceptions to the *State Management Layer*.

`mes_user_service.dart`

The class declaration and *fetch users* method send an authenticated *HTTP GET* request to the `/mesUsers` endpoint and deserialise the response into a list of `MesUser` model objects. The *create user* method sends a *HTTP POST* request to the `/createMesUser` endpoint with the new user's role and employee reference in the request body. The *generate password* method and associated response deserialisers handle the password generation endpoint and map the *JSON* result into the `GeneratedPassword` model.

### State Management Layer using Provider

The *Provider* pattern is implemented to ensure reactive UI updates and centralised data handling. Two providers were implemented: `AuthProvider`, which manages authentication state, and `MesUserProvider`, which handles MES user data. Each provider wraps the corresponding service layer calls and exposes reactive state (authenticated user, user list, loading flag, error message) consumed by the *UI Layer* via `context.watch`.

### User Interface

The *Login Page*, shown in Figure 3.12, is the application entry screen presented to all users on launch. It contains *User ID* and *Password* input fields and a *Login* button. Successful authentication redirects the user to the role-appropriate dashboard.

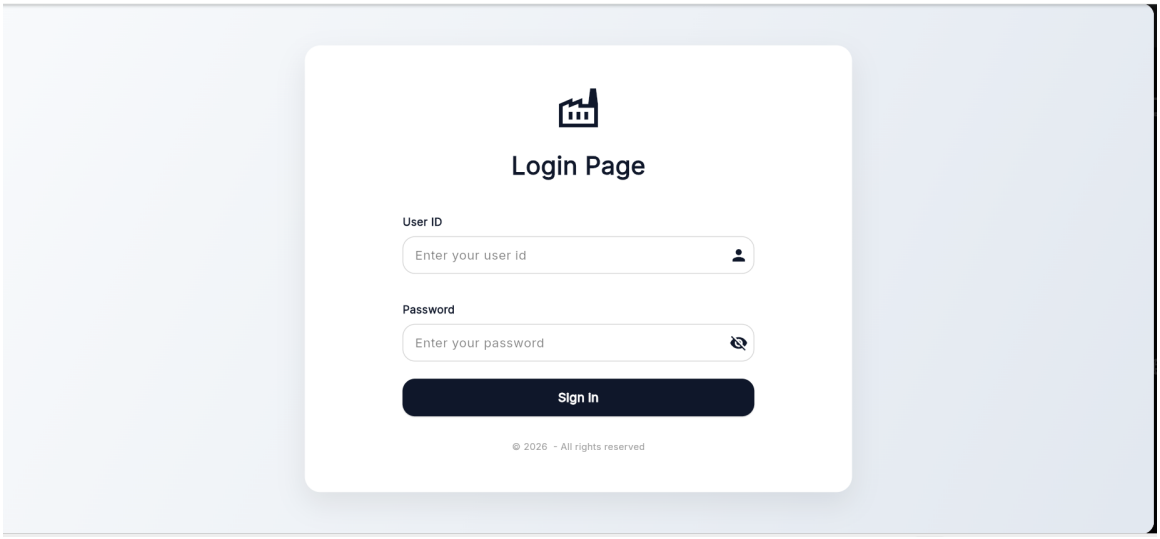


**Figure 3.12.** *Login Page.*

The *MES User List Page*, shown in Figure 3.13, is the Administrator dashboard screen listing all registered *MES* users with their assigned roles and active status, providing action buttons to assign roles or generate passwords for each employee.

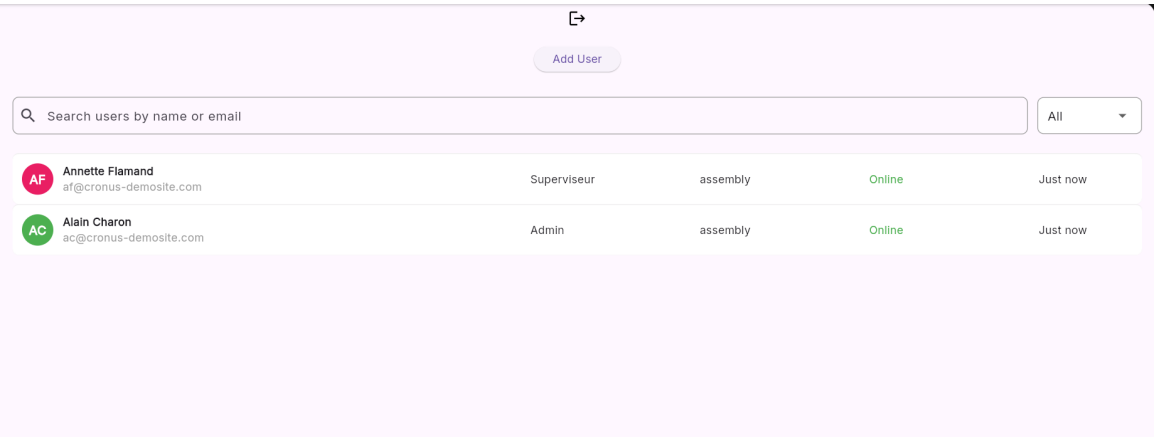


**Figure 3.13.** *MES User List Page*.

The *Assign MES Role Dialog*, shown in Figure 3.14, is a modal dialog allowing the Administrator to select a Business Central employee record and assign them an *MES* role (Operator or Supervisor), creating a new `MES User` record via the `/createMesUser` endpoint.

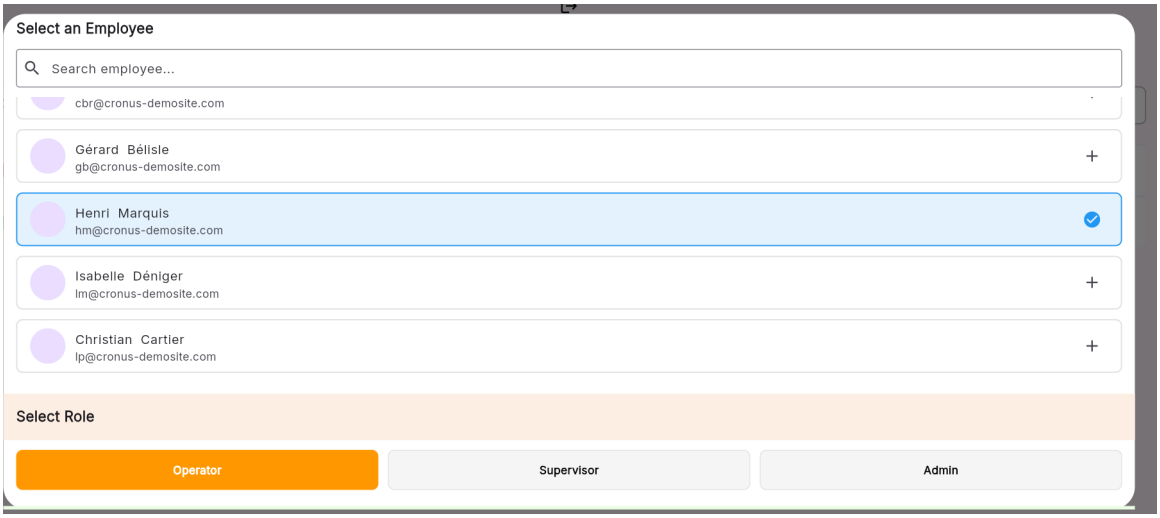


**Figure 3.14.** *Assign MES Role Dialog*.

The *Generate Password Dialog*, shown in Figure 3.15, is a modal dialog through which the Administrator triggers server-side secure password generation for a selected *MES* user. The generated password is displayed once and must be communicated to the user out-of-band.

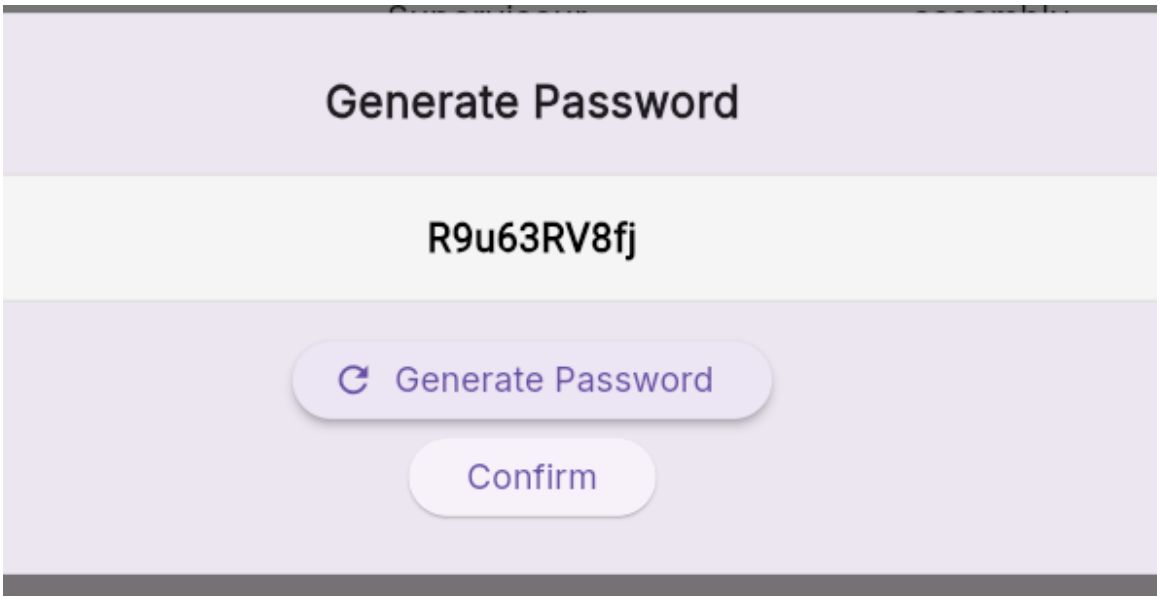


**Figure 3.15.** *Generate Password Dialog*.

The *Password Generated Successfully Dialog*, shown in Figure 3.16, is a confirmation dialog showing the plaintext password to the Administrator for one-time transmission to the user. The password is not stored in plaintext on the server after this point.

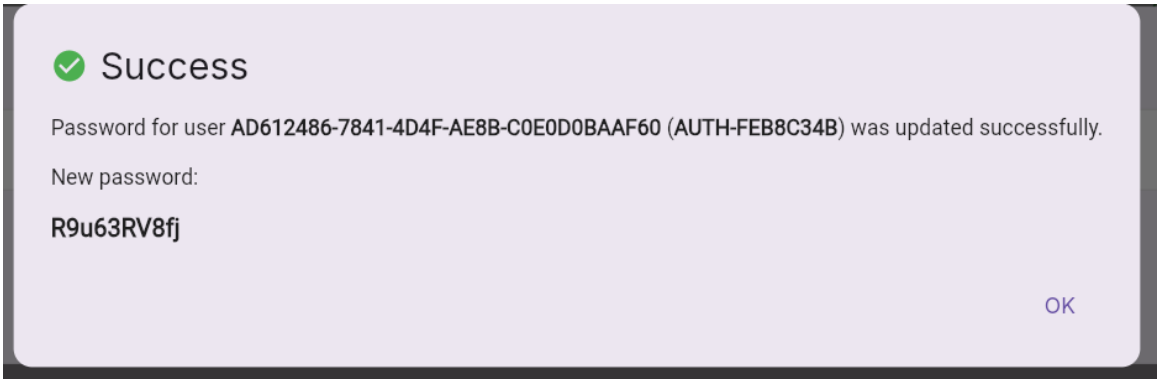


**Figure 3.16.** *Password Generated Successfully Dialog*.

The *Change Password Page*, shown in Figure 3.17, is presented to users whose *needChangePassword* flag is `true` after login, requiring them to enter their current password and choose a new password satisfying the rules defined in `BR-AUTH-003` before accessing the main application.

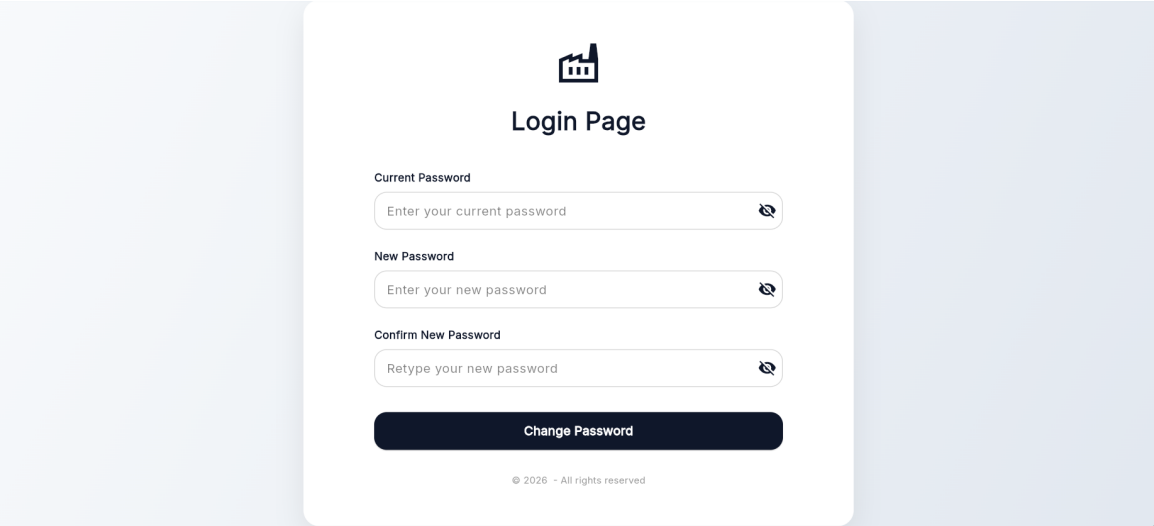


**Figure 3.17.** *Change Password Page.*

The *Change Role Dialog*, shown in Figure 3.18, is a modal dialog through which the Administrator updates the role and work-centre assignment of an existing *MES* user. The dialog pre-populates the current role and presents a three-button row (Operator / Supervisor / Admin); selecting Admin hides the work-centre list, selecting Operator shows a single-select list, and selecting Supervisor shows a multi-select list. An inline error banner displays any server-side validation failure. Tapping *Save Changes* submits the update and revokes all active tokens for the affected user.



**Figure 3.18.** *Change Role Dialog.*

The user row in the *MES User List Page* now includes a `PopupMenuButton` action menu accessible via a trailing icon on each row. The menu exposes two actions: *Change Role / Department*, which opens the ChangeRoleDialog described above, and *Activate* or *Deactivate*, whose label switches dynamically based on the user's current `isActive` flag. Selecting *Deactivate* immediately revokes all active sessions for that account; selecting *Activate* re-enables login.

## Conclusion

This sprint successfully delivered a complete authentication and user management foundation for the *MES* application. The backend, built in Microsoft Dynamics Business Central using AL, implements secure credential validation, *PBKDF2-SHA256* password hashing, session token management with automatic expiry, and a full set of *RESTful* API endpoints exposed through MES Web Service (codeunit 50126). The Flutter frontend provides a clean, role-aware user experience covering login, logout, forced password change, administrator-level user creation, role and work-centre management, and account activation control.

All business rules defined at the start of the sprint — credential validation (`BR-AUTH-001`), session management (`BR-AUTH-002`), password security (`BR-AUTH-003`), role–work-centre constraint (`BR-ADMIN-001`), and token revocation on role change (`BR-ADMIN-002`) — have been implemented and verified. The layered architecture adopted on the frontend ensures that the codebase remains maintainable and extensible as new sprints introduce additional *MES* features.

# Chapter 4

# MES Sprint 2 Repport

## Contents

## Introduction

This sprint covers **Machine Monitoring, Production Order Management, and Operation History** in the *MES* application, giving Operators and Supervisors real-time visibility into machine states and a full audit trail of completed operations.

It addresses three related sub-domains: live machine status tracking from Business Central to the Flutter frontend, production order management with operation lifecycle initiation, and a history view showing all completed and cancelled operations per machine.

## 4.1 Sprint Backlog

This sprint covers **Domain 2:** ***Machine Monitoring, Production Order Management & History***.

**Table 4.1.** Sprint Backlog

| ID | User Story | Tasks |
|---|---|---|
| **US-2.1** | As an Operator or Supervisor, I need to see the list of machines in my work centre with their current status so that I can see which machines are currently active. | *Business Central (AL):*<br>• Created table MES MachineStatus.<br>• Created enum MES Machine Status.<br>• Created function `FetchMachines()` in codeunit MES Machine Fetch.<br>• Exposed `FetchMachines()` through codeunit MES Web Service.<br>• Created page MES Machine List.<br>*Flutter:*<br>• Created model MachineModel.<br>• Created service MESMachineListService to consume the `FetchMachines()` API.<br>• Created provider MesMachinesProvider.<br>• Created page MachineListPage.<br>• Created widget MachineCard.<br>• Implemented automatic machine list refresh in MESMachineListService. |
| **US-2.2** | As an Operator or Supervisor, I need to view the production orders assigned to a machine so that I know what work needs to be done. | *Business Central (AL):*<br>• Created function `getMachineOrders()` in codeunit MES Machine Fetch.<br>• exposed `getMachineOrders()` in codeunit MES Web Service. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| | | *Flutter:*<br>• Created model MachineOrderModel.<br>• Created service ErpMachineOrdersService to consume the `getMachineOrders()` API.<br>• Created provider MachineordersProvider.<br>• Created page MachineOrderPage.<br>• Created widget OrderCard.<br>• Created model BadgeStyle. |
| **US-2.3** | As an Operator or Supervisor, I need to start a production operation so that the system records that the order has begun. | *Business Central (AL):*<br>• Created table MES Operation state.<br>• Created enum MES Operation Status.<br>• Created function `TryStartOperation()` in codeunit MES Machine Validation.<br>• Created function `InsertMESOperation()` in codeunit MES Machine Write.<br>• Exposed `InsertMESOperation()` through codeunit MES Web Service.<br>• Created page MES Operations.<br>*Flutter:*<br>• Implemented ErpMachineOrdersService that Consumes the `startOperation` API.<br>• Implement MachineordersProvider to manage its state.<br>• Implemented start operation action handling in ActionButtons.<br>• Implemented automatic navigation to OrdersProgressionPage |
| **US-2.4** | As a Supervisor or Operator, I need to monitor the current status of all active operations on a machine so that I can follow the progress of each order. | *Business Central (AL):*<br>• Created function `fetchMachineOperationStatus()` in codeunit MES Machine Fetch.<br>• Exposed `fetchMachineOperationStatus()` through codeunit MES Web Service. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| | | *Flutter:*<br>• Updated ErpMachineOrdersServiceto Consume the `fetchMachineOperationStatus`.<br>• Created model OperationStatusStyle.<br>• Created page OrdersProgressionPage.<br>• Created the widgets OperationCard, OperationStatusBadge, widget OperationInfoGrid, and OperationProgressBar.<br>• Implemented automatic refresh for operation status monitoring. |
| **US-2.5** | As an Operator or Supervisor, I need a tabbed view for a machine so that I can navigate between pending orders and active operation progress without leaving the machine context. | *Flutter:*<br>• Created page MachineMainPage.<br>• Created the widgets TopNavigationBar and NavButton.<br>• Implemented tab navigation in MachineMainPage.<br>• Implemented navigation from MachineCard to MachineMainPage. |
| **US-2.6** | As an Operator or Supervisor, I need to filter and sort production orders so that I can quickly find the most relevant work. | *Flutter:*<br>• Created widget GlobalSearchBar.<br>• Implemented order search, filtering, and sorting in MachineOrderPage. |
| **US-2.7** | As a Supervisor or Operator, I need to view the history of completed operations for a machine so that I can audit past production. | *Business Central (AL):*<br>• Created function `fetchOperationsStatusAndProgress()` in codeunit MES Machine Fetch.<br>• Exposed `fetchOperationsStatusAndProgress()` in codeunit MES Web Service. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| | | *Flutter:*<br>• Updated ErpMachineOrder-sService to consume the `fetchMachineOperationStatusAndProgress` API.<br>• Updated MachineordersProvider to manage its state.<br>• Created page MachineHistoryPage.<br>• Created the widgets HistoryCard, HistoryBadge-AndId, and HistoryInfoGrid.<br>• Implemented history search and sorting in MachineHistoryPage.<br>• Implemented navigation from HistoryCard to OperationDetailPage. |

## 4.2 Business Rules

**Table 4.2.** Business Rules

| ID: Business Rule | Description |
|---|---|
| **Machine Status Rules** | |
| `BR-MCH-001`: Machine Status Tracking | • Machine status is stored as an append-only log in the `MES Machine Status` table; the latest record per machine represents the current state.<br>• Valid statuses are *Idle*, *Starting*, and *OutOfOrder*.<br>• The Flutter frontend polls the status endpoint every 5 seconds via a *Stream* to maintain a live view. |
| `BR-MCH-002`: Work Centre Scoping | • The machine list is always filtered to the work centre stored in the authenticated user's session token response.<br>• An Operator or Supervisor can only see machines belonging to their assigned work centre. |
| **Operation Lifecycle Rules** | |

Continued on next page

| ID: Business Rule | Description |
|---|---|
| `BR-OPS-001`: Start Operation Validation | • Only routing lines with status *Released* may be started.<br>• An operation that already has a *Running* record may not be started again.<br>• A machine may not run more than one operation at a time; attempting to start a second operation on a busy machine is rejected with a descriptive error message. |
| `BR-OPS-002`: Immutable Operation History | • Each state transition (start, pause, resume) inserts a *new* record into `MES Operation Status` rather than modifying an existing one, preserving a full audit trail.<br>• The latest status for a given (machine, order, operation) triple is determined by sorting on `Last Updated At` descending and taking the first record. |
| `BR-OPS-003`: Active Operation Query | • When fetching active operations for a machine, the backend groups records by (production order, operation) and returns only those whose most recent record has status *Running* or *Paused*.<br>• Operations whose latest record is *Finished* are excluded from the active list. |
| **History Query Rules** | |
| `BR-HIST-001`: History Scope | • The History tab queries `fetchOperationsStatusAndProgress` with `fetchFinished = true`, returning only *Finished* and *Cancelled* executions.<br>• The earliest *Running* status timestamp is resolved as `startDateTime`; the first *Finished* or *Cancelled* timestamp as `endDateTime`.<br>• History is loaded as a one-shot `Future` on page open; it is not polled, as completed records are immutable. |

## 4.3 Design

### 4.3.1 Use Case Diagram

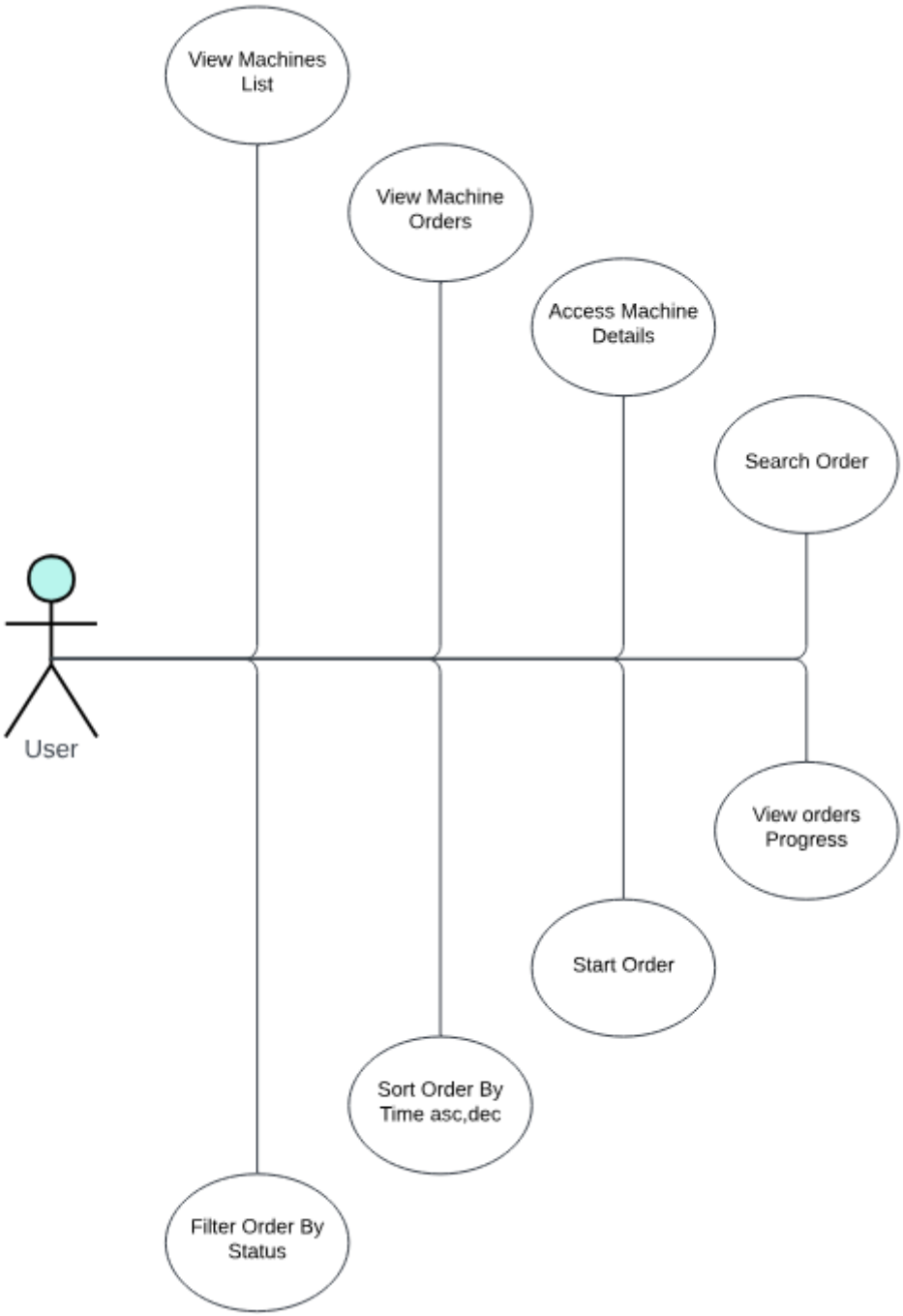


**Figure 4.1.** UML Use Case Diagram — Machine & Production Management.

The UML Use Case Diagram (Figure 4.1) shows the two primary actors (Operator and Supervisor) and their interactions with the machine monitoring, production order management, and history subsystem. In addition to viewing machines, browsing orders, and managing operation

lifecycle transitions, the diagram includes the *View Operation History* use case, which allows both actors to access the completed and cancelled operation log for any machine in their work centre.

### 4.3.2 Textual Use Case Description

The following table (Table 4.3) provides the detailed textual description of the *Start Operation* use case.

**Table 4.3.** Textual Use Case Description — Start Operation

| | |
|---|---|
| **Use Case Name** | Start Operation |
| **Primary Actor** | Operator |
| **Preconditions** | • The Operator is authenticated and has a valid session token.<br>• The target routing line has status *Released*.<br>• No other operation is currently *Running* on the same machine.<br>• The operation has not already been started (no existing *Running* record for this order/operation pair). |
| **Postconditions** | • A new `MES Operation Status` record is inserted with status *Running* and the current *UTC* timestamp.<br>• The *Operations Progress* tab reflects the new running operation within the next polling cycle.<br>• If any precondition fails, a descriptive error dialog is shown to the operator and no record is inserted. |
| **Main Scenario** | 1. The Operator navigates to the *Machine List Page* and selects a machine.<br>2. The system displays the *Machine Orders Page* listing all applicable production orders.<br>3. The Operator locates a *Released* order and taps *Start Order*.<br>4. The frontend sends a *POST* request to `MESMachinesActionsEndpoints_startOperation`.<br>5. The backend validates all preconditions (`BR-OPS-001`).<br>6. On success, a `MES Operation Status` record is inserted.<br>7. The frontend navigates to the *Orders Progression Page*. |

### 4.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for each machine and opera-

tion flow.

Figure 4.2 shows the fetch machines flow. Figure 4.3 illustrates the get machine orders flow. Figure 4.4 depicts the start operation flow. Figure **??** presents the fetch operations status flow.

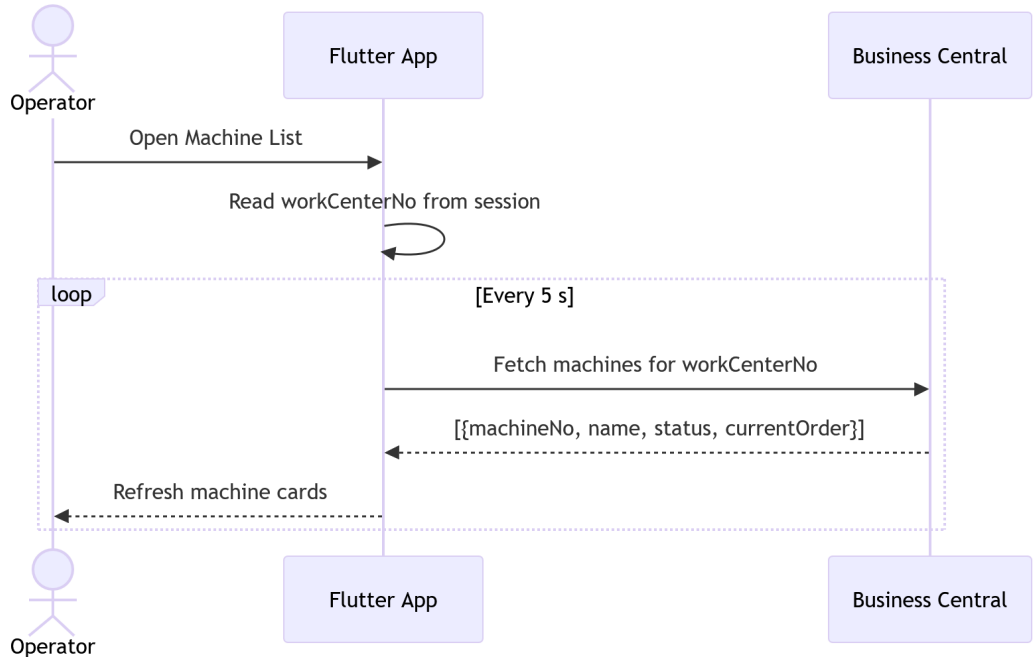


**Figure 4.2.** Fetch Machines — Sequence Diagram.

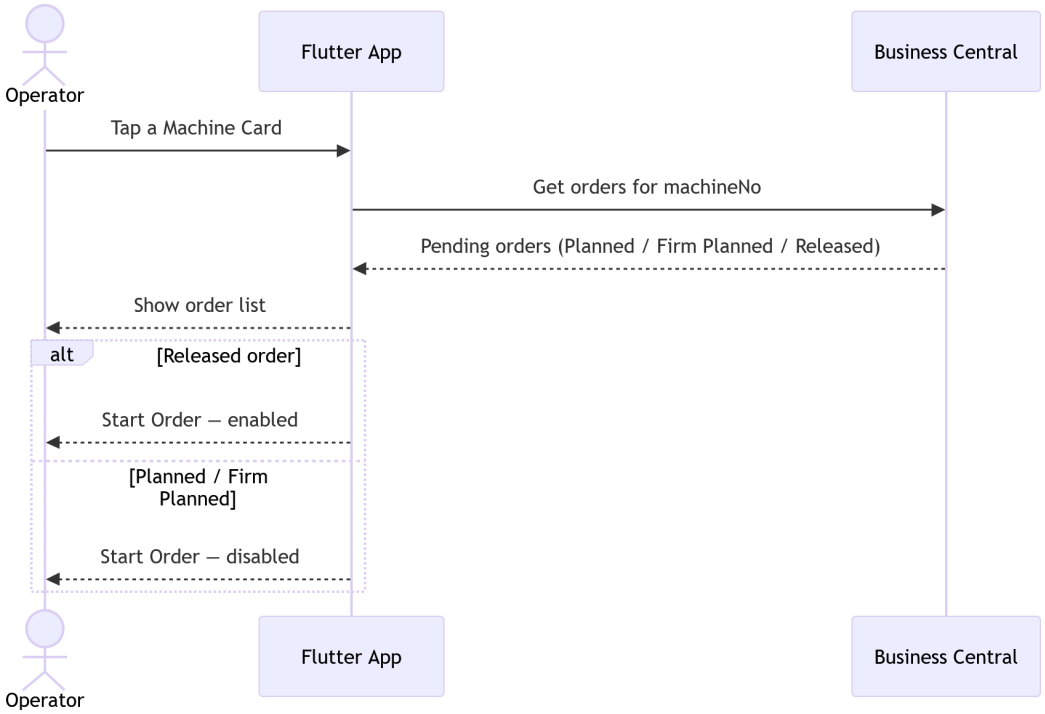


**Figure 4.3.** Get Machine Orders — Sequence Diagram.

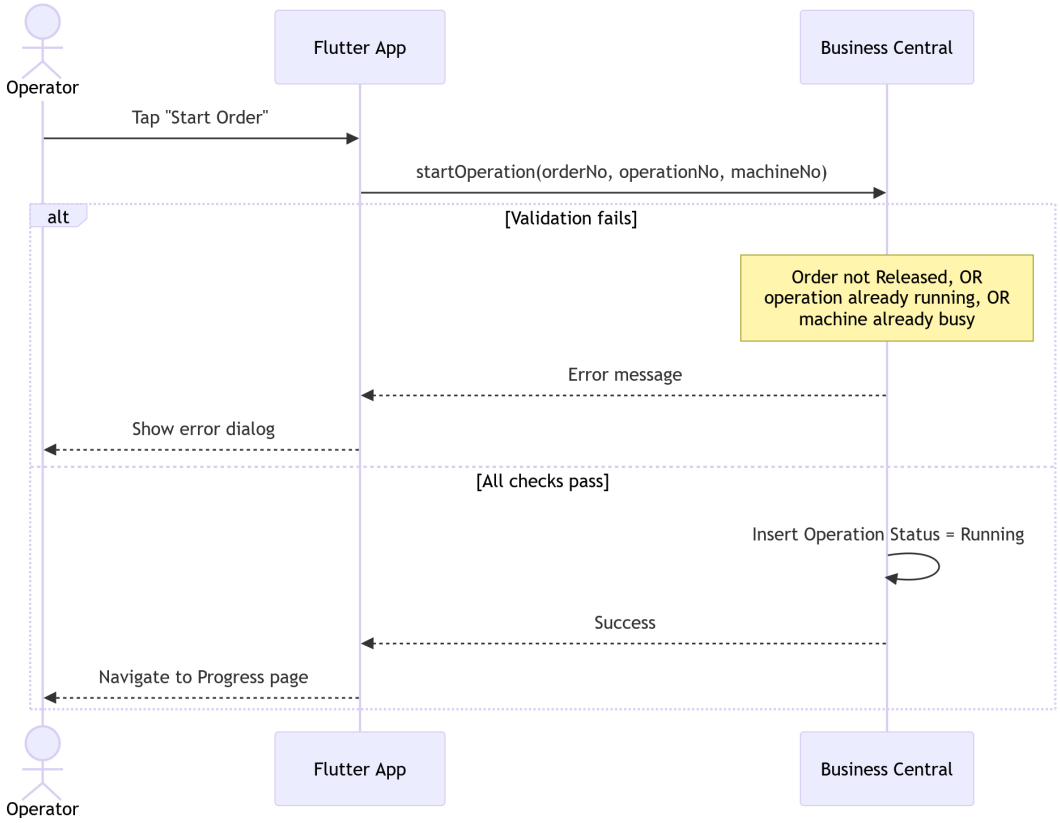


**Figure 4.4.** Start Operation — Sequence Diagram.



**Figure 4.5.** Fetch Operation History — Sequence Diagram.

Figure 4.5 shows the history retrieval sequence. The Flutter frontend calls the same `fetchOperationsStatusAndProgress` endpoint used by the active monitoring view, but with `fetchFinished = true`. The Business Central backend returns only executions whose latest status record is *Finished* or *Cancelled*, along with resolved start and end timestamps.

### 4.3.4 Class Diagram

The UML Class Diagram (Figure 4.6) of the machine and production management domain shows the `MES Machine Status` and `MES Operation Status` tables and their relationships with the Business Central standard tables `Machine Center` and `Prod. Order Routing Line`.

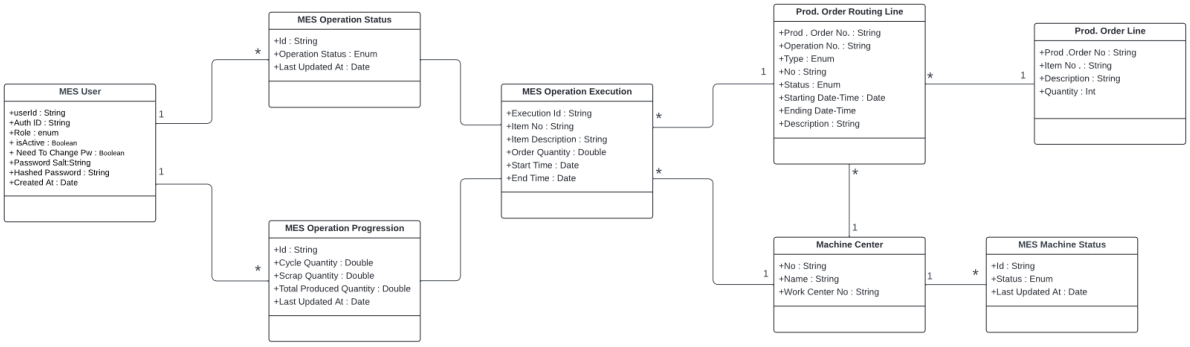


**Figure 4.6.** UML Class Diagram — Machine & Production Management.

## 4.4 Implementation

### 4.4.1 Backend Implementation

#### Backend Structure Overview

The machines domain follows the same layered structure established in Sprint 1. Two new sub-folders were added under `src/1-Tables/machines/` and `src/2-Enum/machines/`, and a new codeunit was placed under `src/3-CodeUnit/machines/`. The OData web service is published through the existing `WebServices.xml` mechanism under a new service name.

#### MES Machine Status Table

The `MES Machine Status` table (50107) stores the full history of machine state changes as an append-only log. Each insert creates a new record; the most recent record per machine represents the current state. Table 4.4 lists the fields and their descriptions.

**Table 4.4.** MES Machine Status (Table 50107) — Field Dictionary

| Name | Type | Description |
|---|---|---|
| Id | Code[50] | Primary key. GUID-derived value assigned automatically on insert if blank. |
| Machine No. | Code[20] | Foreign key to `Machine Center."No."`. Identifies the machine this status record belongs to. |
| Status | Enum (`MES Machine Status`) | Current machine state: *Idle*, *Starting*, or *OutOfOrder*. |
| Current Prod. Order No. | Code[20] | Foreign key to `Prod. Order Routing Line."Prod. Order No."`. Records which production order the machine is currently executing, if any. |
| Last Updated At | DateTime | UTC timestamp set automatically on insert. Used to find the most recent status record for a given machine. |

### MES Operation Status Table

The `MES Operation Status` table (50108) implements the same append-only pattern for operation lifecycle events. Every state transition (start, pause, resume, finish) inserts a new record; queries always sort by `Last Updated At` descending to obtain the current state. Table 4.5 provides the full field dictionary.

**Table 4.5.** MES Operation State (Table 50108) — Field Dictionary

| Name | Type | Description |
|---|---|---|
| Id | Code[50] | Primary key. GUID-derived identifier assigned automatically on insert. |
| Execution Id | Code[50] | Foreign key to `MESOperationExecution`. |
| Operator Id | Code[50] | Foreign key to `MES User."User Id"`. The operator who triggered the state transition. |
| Operation Status | Enum (`MES Operation Status`) | Lifecycle state at the time of this record: *Running*, *Paused*, or *Finished*. |

Continued on next page

| Name | Type | Description |
|---|---|---|
| Last Updated At | DateTime | UTC timestamp set automatically on insert. Together with the composite key (Machine No, Prod Order No, Operation No), this field is used to retrieve the current status via descending sort. |

Key points:

- Both tables use an append-only design: no `Modify` is called on existing records.
- The *secondary index* `ExecutionTimeline("Execution Id", "Last Updated At")` on `MES Operation State` supports efficient retrieval of the latest event per group.

**Enumerations**

Two enumerations support the domain. `MES Machine Status` (Enum 50101) defines three values: *Idle*, *Starting*, and *OutOfOrder*. `MES Operation Status` (Enum 50102) defines three values: *Running*, *Paused*, and *Finished*. Both enumerations are declared `Extensible = true` to allow future additions without modifying the base objects.

**Machine Actions Business Logic**

All machine and operation business logic is encapsulated in codeunit 50130 `MES Machine Actions`. The codeunit exposes four public procedures that are called directly by the OData web service layer.

**FetchMachines.** Accepts a *work centre number* and returns a *JSON* array of machine objects. For each `Machine Center` record scoped to the given work centre, the procedure looks up the latest `MES Machine Status` record via `FindLast()` and inlines the current status and active production order number into the response object. If no status record exists, the machine is reported as *Idle*.

**getMachineOrders.** Accepts a machine number and returns a *JSON* array of production routing lines whose status is *Planned*, *Firm Planned*, or *Released* and whose type is *Machine Center*. Routing lines that already have an existing `MES Operation Status` record are excluded from the result, ensuring that only work not yet started is shown in the order list. For each qualifying line, the corresponding `Prod. Order Line` is joined to supply item description and order quantity.

**startOperation.** Accepts a production order number, operation number, and machine number. The procedure delegates precondition checking to a `[TryFunction]` (`TryStartOperation`) that enforces `BR-OPS-001`: it verifies the routing line is *Released*, confirms no *Running* record exists for the same order/operation, and checks that the machine is not already busy with another

operation. If all checks pass, a new `MES Operation Status` record with status *Running* is inserted by `InsertMESOperation`. On failure, the last error text is captured and returned as a structured *JSON* error response.

**fetchMachineOperationStatus.** Returns the current active operations for a machine as a *JSON* array. The procedure sets a descending composite key on (Machine No, Prod Order No, Operation No, Last Updated At) and iterates the result set, emitting at most one entry per (order, operation) pair — the one with the most recent timestamp. Only entries whose latest status is *Running* or *Paused* are included in the output, satisfying `BR-OPS-003`.

### 4.4.2 REST API and Web Services Implementation

Communication between Flutter and Business Central for the machines domain uses the same *OData V4 Unbound Actions* pattern established in Sprint 1. A new web service entry was added to `WebServices.xml` exposing codeunit 50130 under the service name `MESMachinesActionsEndpoints`. After deployment, the four procedures are reachable at the following endpoints:

- `POST .../ODataV4/MESWebService_FetchMachines`
- `POST .../ODataV4/MESWebService_getMachineOrders`
- `POST .../ODataV4/MESWebService_startOperation`
- `POST .../ODataV4/MESWebService_fetchMachineOperationStatus`

All endpoints follow the same *JSON* request/response convention used by the authentication layer: the request body carries named parameters as top-level *JSON* fields, and the response wraps the AL return value inside a `"value"` field which the Flutter service layer double-decodes.

### 4.4.3 Frontend Implementation

The machines domain retains the same four-layer call stack introduced in Sprint 1 (*Model* → *Service* → *Provider* → *UI*), but Sprint 2 introduced a significant reorganisation of the *UI layer* file structure, making the codebase easier to maintain and facilitating code reuse across components.

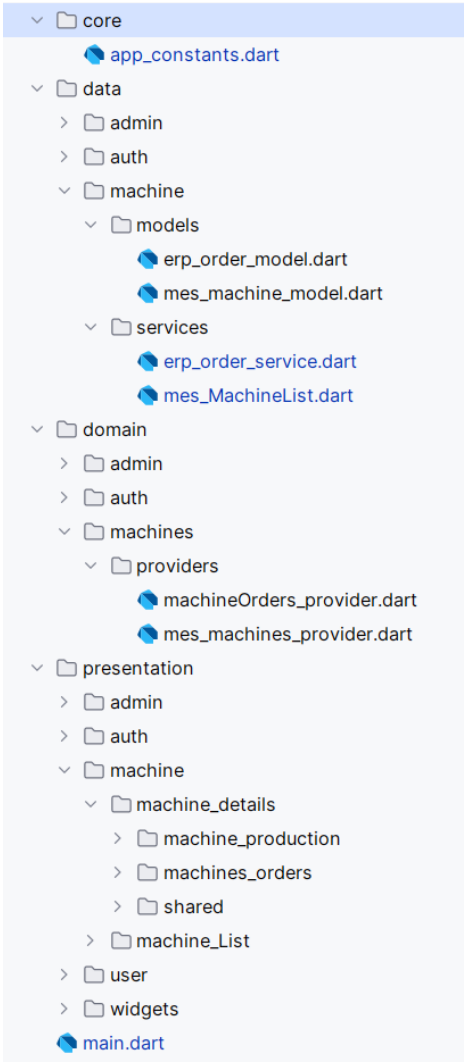


**Figure 4.7.** Flutter machines domain — layered architecture.

#### Model Layer

Two models were introduced. `MachineModel` (`mes_machine_model.dart`) holds the machine number, name, current status string, and active order number, populated from the `FetchMachines` response. `MachineOrderModel` (`erp_order_model.dart`) holds the full set of production order fields returned by `getMachineOrders`, including order number, status, operation number, planned start and end *DateTime* values, item number, item description, order quantity, and operation description.

#### Service Layer

Two service classes handle all network communication for the machines domain. `MESMachineListService` (`mes_MachineList.dart`) exposes a `fetchMachines` method that posts the work centre number and parses the double-encoded *JSON* array into a list of `MachineModel` objects. It also exposes a `streamMachines` method that wraps `fetchMachines` in an `async*` generator loop with a 5-second `Future.delayed` delay, emitting an updated list on every cycle and yielding an empty list on error to prevent the *StreamBuilder* from entering an error state.

`ErpMachineOrdersService` (`erp_order_service.dart`) exposes three methods: `getMachineOrders` for fetching the order list, `getStartOperationValidation` for triggering the start operation endpoint and parsing the success/failure result, and `fetchMachineOperationStatus` for retrieving active operation records. A `streamMachines` generator on this service applies the same 5-second polling pattern for the operations progress view.

### State Management Layer

Two providers were introduced. `MesMachinesProvider` (`mes_machines_provider.dart`) is a plain (non-`ChangeNotifier`) provider that delegates directly to the `MESMachineListService` stream. Because all state is carried by the stream itself, no `notifyListeners` mechanism is needed and the *StreamBuilder* in the UI subscribes directly to the emitted values.

`MachineordersProvider` (`machineOrders_provider.dart`) wraps both the order list fetching and the operation start flow with the standard `ChangeNotifier` pattern. It exposes `getMachineOrders` which populates a `machineOrders` list, and `startOrder` which calls the service and returns a boolean success flag. It also exposes `getMachineOperationsStatusStream` which passes the polling stream through directly to the UI layer.

### User Interface

The *Machine List Page* is the entry point for operators after login. It displays a live-updating grid (tablet/desktop) or list (mobile) of machine cards scoped to the authenticated work centre. Each card shows the machine name, its current status with a colour-coded badge, and the active production order number. The layout adapts across breakpoints: a single-column list below 600 dp, a two-column grid below 1024 dp, and a four-column grid above. As illustrated in Figure 4.8, the interface is consistent across both desktop and mobile platforms.

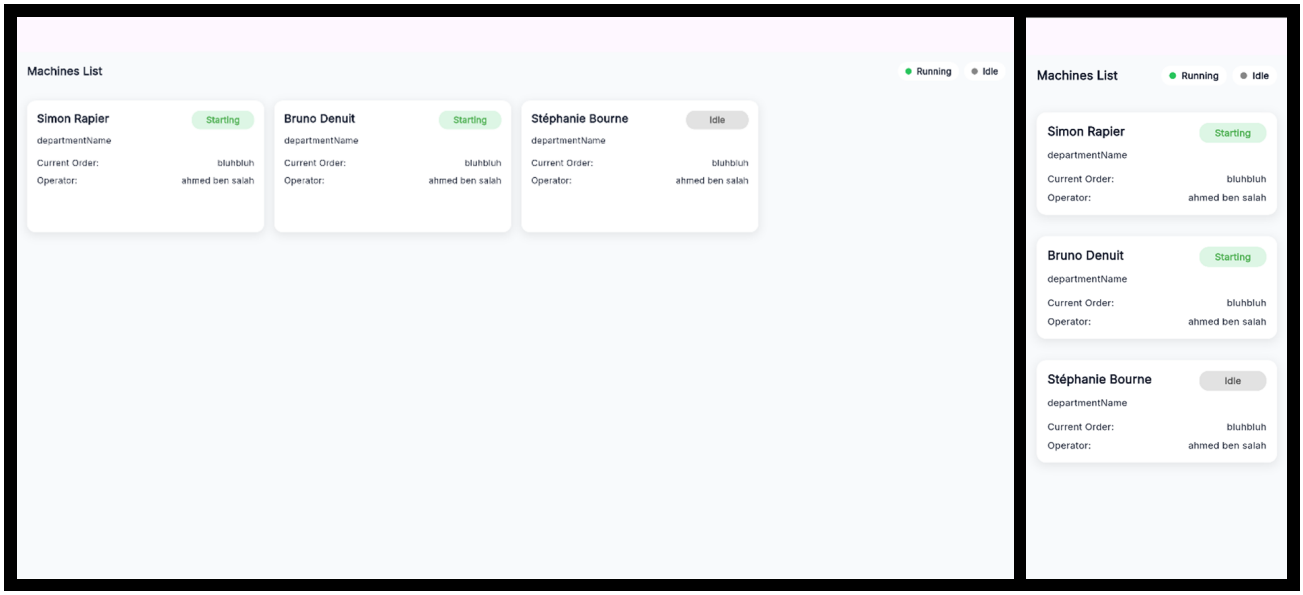


**Figure 4.8.** *Machine List Page* — desktop and mobile views.

The user interface for the machines domain is split across two top-level pages accessible via a shared *Top Navigation Bar* with tabs for *Orders*, *Progress*, *Consumable*, and *History*.

The *Machine Orders Page* displays the production routing lines assigned to the selected machine. A *Global Search Bar* allows filtering by free text (order number, item description, or

date), by status, and sorting by planned start date. Order cards use a responsive layout that switches between a stacked *narrow layout* and a side-by-side *wide layout* at a 520 dp breakpoint. Only *Released* orders display an active *Start Order* button; *Planned* and *Firm Planned* orders show it disabled. Figure 4.9 illustrates both the desktop and mobile views.

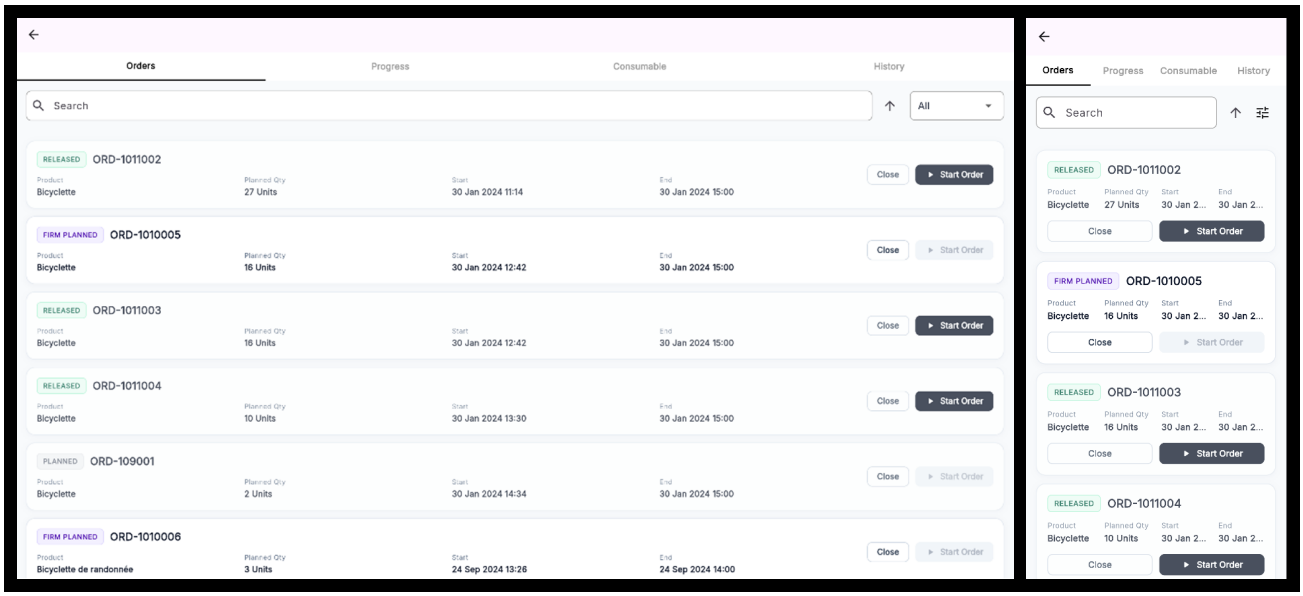


**Figure 4.9.** *Machine Orders Page* — desktop and mobile views.

The *Orders Progression Page* displays the currently active operations on a machine, polling every 5 seconds. Each operation card shows a colour-coded status badge (green for *Running*, amber for *Paused*), the order and operation identifiers, the last updated timestamp, and a progress bar. The card layout also switches between narrow and wide variants at the 520 dp breakpoint. *Pause* and *Resume* action buttons adapt their colour and label to the current operation status, as shown in Figure 4.10.

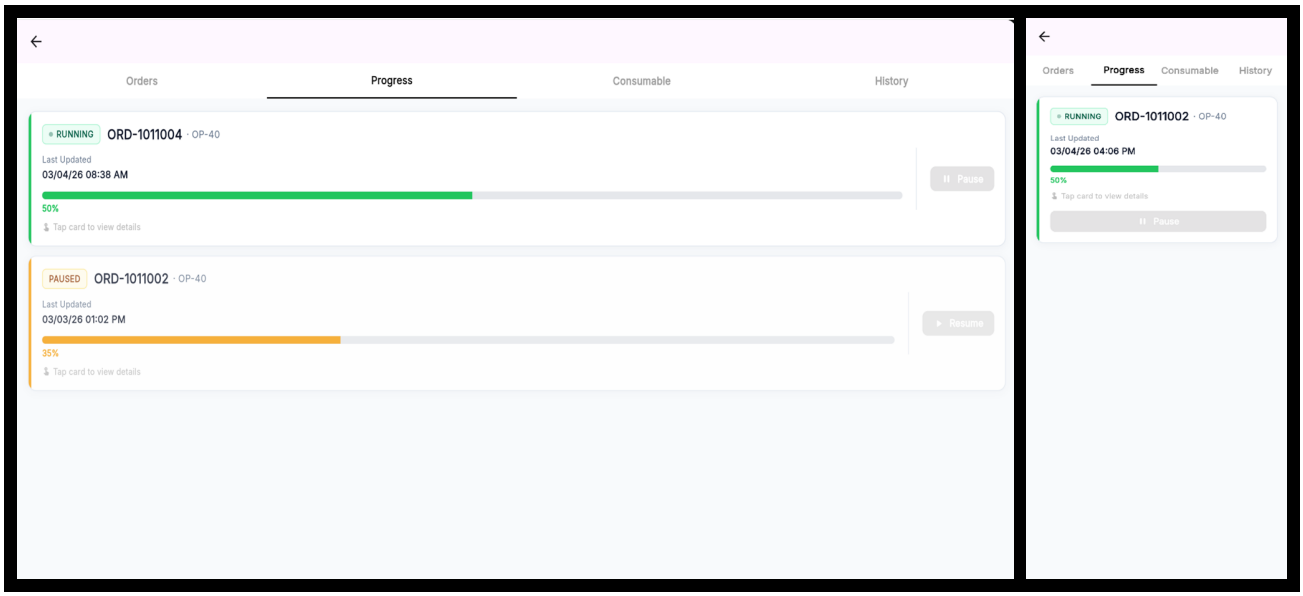


**Figure 4.10.** *Orders Progression Page* — desktop and mobile views.

The *Machine History Page* (Figure 4.11) displays completed operations assigned to the selected machine, limited to *Finished* and *Cancelled* statuses. It shares the same layout and filtering behaviour as the *Machine Orders Page* — including free-text search, status filtering, and sort controls — but omits the *Start Order* button. Tapping an operation card navigates to its *Operation Detail Page* for review. The *Machine History Page* is accessible via the *History* tab in the TopNavigationBar. Figure 4.11 shows the desktop and mobile views.

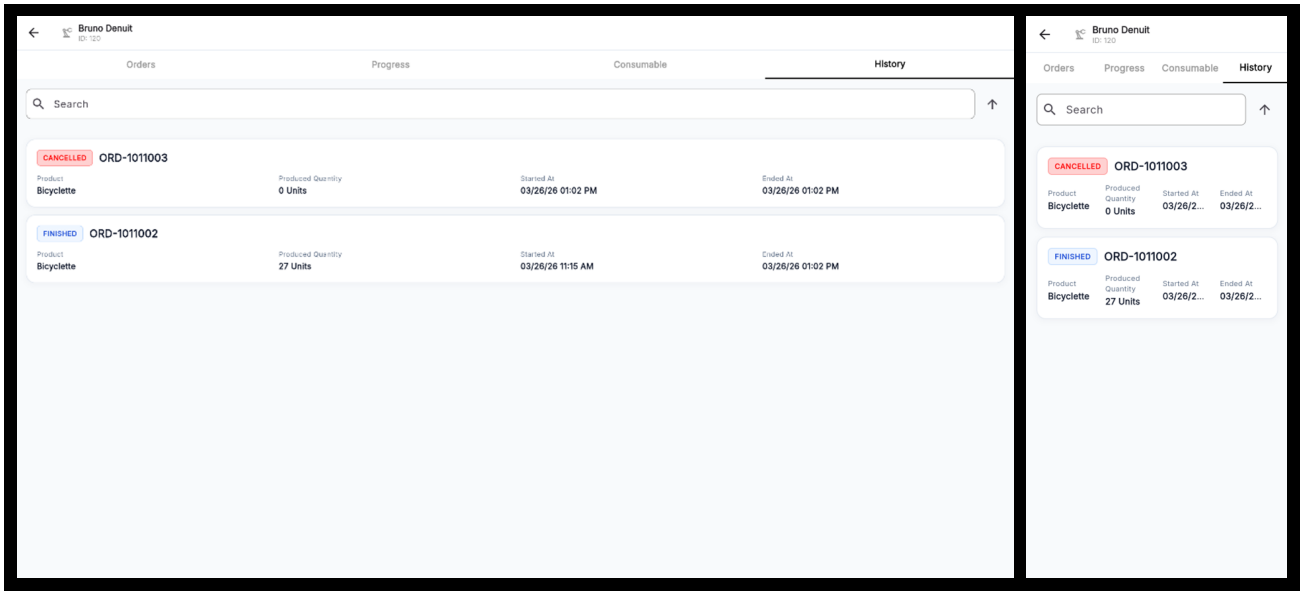


**Figure 4.11.** *Machine History Page* — desktop and mobile views.

## Conclusion

This sprint successfully delivered the machine monitoring and production order management foundation of the *MES* application. On the backend, two new append-only tables (`MES Machine Status` and `MES Operation Status`) and a dedicated codeunit (`MES Machine Actions`) were implemented in Business Central AL, exposing four *OData V4* endpoints that cover the full machine and operation lifecycle. The append-only design ensures a complete, tamper-evident audit trail of every state transition without requiring any record modification.

All business rules defined at the start of the sprint have been enforced: work centre scoping (`BR-MCH-002`), start operation validation (`BR-OPS-001`), immutable history (`BR-OPS-002`), active operation querying (`BR-OPS-003`), and history scope (`BR-HIST-001`). The Flutter frontend delivers a responsive, real-time interface with 5-second polling streams for the machine list and operations progress view, and a one-shot history load for the completed operation audit log. The codebase is now ready to extend in Sprint 3 with consumable tracking, operation finishing, and production declaration.

# Chapter 5

# MES Sprint 3 Repport

## Contents

## Introduction

This sprint report covers **Domain 3: Production Execution & Traceability**. Building upon the authentication and machine monitoring infrastructure established in the previous sprints, this sprint delivers the core production execution features: declaring production output, pausing and resuming operations, finishing or cancelling orders, tracking production cycles over time, monitoring Bill of Materials consumption, component barcode scanning, scrap declaration, supervisor proxy declaration, and label printing. The result is a fully operational shop-floor execution loop from order assignment through to order closure, material traceability, and printed part identification.

## 5.1 Sprint Backlog

This sprint covers **Domain 3: *Production Execution & Traceability***.

**Table 5.1.** US-3.1 — Declare Production

| ID | User Story | Acceptance Criteria |
|---|---|---|
| **US-3.1** | As an Operator or Supervisor, I need to declare the quantity of parts produced in a cycle so that the system records my production progress. | *Business Central (AL):*<br>• Created table MES Operation Progression.<br>• Created the functions:<br>– `TryDeclareProduction()` in codeunit MESMachineValidation.<br>– `InsertNewProgressionCycle()` in codeunit MESMachineInsert.<br>– `declareProduction()` in codeunit MESMachineWrite.<br>• Exposed `declareProduction()` through the codeunit MES Web Service.<br>• Created page MES Operation Progression list page.<br>*Flutter:*<br>• Created ErpMachineOrdersService to consume the `declareProduction` API and MachineordersProvider to manage it.<br>• Created widget DeclareProductionDialog.<br>• Added the `Declare Production` action in ActionButtonsContainer. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| **US-3.2** | As an Operator or Supervisor, I need to pause and resume an active operation so that production interruptions are tracked accurately. | *Business Central (AL):*<br>• Created the functions:<br>– `TryPauseOperation()` and `TryResumeOperation()` in codeunit MESMachineValidation.<br>– `ExecuteOperationTransition()`, `pauseOperation()`, and `resumeOperation()` in codeunit MESMachineWrite.<br>• exposed `pauseOperation()` and `resumeOperation()` throught codeunit MES Web Service.<br>*Flutter:*<br>• Implemented ErpMachineOrdersService to consume the new endpoints<br>• Implemented MachineordersProvider to manage its state.<br>• Created widget OperationActionButtons.<br>• Implemented pause/resume operation toggle handling in OrdersProgressionPage. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| **US-3.3** | As a Supervisor, I need to finish or cancel a production operation so that the machine is released and the order is closed in the system. | *Business Central (AL):*<br>• Created the functions:<br>– `TryCloseOperation()` in codeunit MESMachineValidation.<br>– `InsertOperationStatus()` and `InsertIdleMachineStatus()` in codeunit MESMachineInsert.<br>– `finishOperation()` and `cancelOperation()` in codeunit MESMachineWrite.<br>• Exposed `finishOperation()` and `cancelOperation()` in codeunit MES Web Service.<br>*Flutter:*<br>• Updated ErpMachineOrdersService to consume the new endpoints.<br>• Updated MachineordersProvider to manage its state.<br>• Updated ActionButtonsContainer to handle finish/cancel operation flow.<br>• Implemented finish/cancel button state logic in ActionButtonsContainer. |
| **US-3.4** | As an Operator or Supervisor, I need a dedicated detail page for an operation so that I can see all real-time information in one place. | *Business Central (AL):*<br>• Created table MES Operation Execution.<br>• Created function `InsertMESOperationExecution()` in codeunit MESMachineInsert.<br>• Created function `fetchOperationLiveData()` in codeunit MESMachineFetch.<br>• Exposed `fetchOperationLiveData()` through codeunit MES Web Service.<br>• Created page MES Operation Execution. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| | | *Flutter:*<br>• Created model OperationStatusAndProgressModel.<br>• Updated ErpMachineOrdersService to consume the new endpoints.<br>• Updated MachineordersProvider to manage its state.<br>• Created page OperationDetailPage.<br>• Created widget OperationAppbar.<br>• Created widget CurrentOrderInfoContainer. |
| **US-3.5** | As an Operator or Supervisor, I need to see all production cycles declared for an operation so that I can review each declaration's quantity, operator, and timestamp. | *Business Central (AL):*<br>• Created function `fetchProductionCycles()` in codeunit MESMachineFetch.<br>• Exposed `fetchProductionCycles()` throught codeunit MES Web Service.<br>*Flutter:*<br>• Created model ProductionCycleModel.<br>• Added `fromJson()` and computed getters to ProductionCycleModel.<br>• updated ErpMachineOrdersService to consume the new endpoints.<br>• Updated MachineordersProvider to manage its state.<br>• Created widget ProductionCycle. |
| **US-3.6** | As an Operator or Supervisor, I need to see a visual chart of production declarations over time so that I can identify trends and performance patterns. | *Flutter:*<br>• Created widget ProductionChart. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| **US-3.7** | As an Operator or Supervisor, I need to see the Bill of Materials for a production operation so that I know which components are required and how much has already been consumed. | *Business Central (AL):*<br>• Created table MES Component Consumption.<br>• Created function `fetchBom()` in codeunit MESMachineFetch.<br>• Exposed `fetchBom()` through codeunit MES Web Service.<br>• Created page MES Component Consumption.<br>*Flutter:*<br>• Created model ComponentConsumptionModel.<br>• Created MesComponentConsumptionService to consume the `fetchBom()` API.<br>• Created MesComponentConsumptionProvider to manage BOM stream and refresh state.<br>• Created widget RequiredComponent.<br>• Implemented BOM streaming and refresh flow in MesComponentConsumptionService and MesComponentConsumptionProvider.<br>• Implemented component status display and operation-based filtering in RequiredComponent. |
| **US-3.8** | As an Operator or Supervisor, I need to report scrapped units with a reason code so that waste is accurately tracked. | *Business Central (AL):*<br>• Created table MES Component Consumption.<br>• Created function `declareScrap()` in codeunit MES Machine Write.<br>• Exposed `declareScrap()` through codeunit MES Machine Write.<br>• Created MES scrap Code API to fetch scrap codes from the standard Business Central `Scrap Code`. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| | | *Flutter:*<br>• Created model MesScrapCode.<br>• Created MesScrapService to consume the scrap code fetch and `declareScrap()` APIs.<br>• Created MesScrapProvider to manage scrap code and scrap declaration state.<br>• Created dialog DeclareScrapDialog. |
| **US-3.9** | As a Supervisor, I need to declare production on behalf of an operator so that output is attributed to the correct person. | *Business Central (AL):*<br>• Added field `Declared By` to transactional tables MES Operation Progression and MES Component Consumption.<br>*Flutter:*<br>• Created widget OperatorSelector.<br>• Updated DeclareProductionDialog to support supervisor declaration on behalf of an operator.<br>• Connected OperatorSelector to MesUserProvider and passed `onBehalfOfUserId` to MachineordersProvider.declareProduction(). |
| **US-3.10** | As an Operator or Supervisor, I need to scan components into an operation so that consumption is recorded against the BOM. | *Business Central (AL):*<br>• Created the function `insertScans()` in codeunit MES Machine Write.<br>• Created the functions `fetchAllItemBarcodes()` and `resolveBarcode()` in codeunit MES Machine Fetch.<br>• Exposed the three function through codeunit MES Web Service. |

Continued on next page

| ID | User Story | Acceptance Criteria |
|---|---|---|
| | | *Flutter:*<br>• Created model ItemBarcodeModel.<br>• Created MesBarcodeService to consume the APIs `fetchAllBarcodes()`, `insertScans()`, and `resolveBarcode()`.<br>• Created MesBarcodeProvider to manage barcode state and API calls.<br>• Created widget ScannerWidget.<br>• Created page BarcodeListPage.<br>• Updated widget RequiredComponent to integrate item scanning, search, filtering, and BOM refresh after scan submission. |
| **US-3.11** | As an Operator or Supervisor, I need to print a label for the active production order so that produced parts are correctly identified. | *Flutter:*<br>• Created page PrintLabelPage.<br>• Updated ActionButtonsContainer to integrate the *Print Label* action.<br>• Implemented label generation, preview, print flow, and orientation toggle in PrintLabelPage. |
| **US-3.12** | As an Operator or Supervisor, I need a label printed for each declared unit so that individual pieces carry traceable information. | *Flutter:*<br>• Created page DeclarationLabelPage.<br>• Implemented declaration label generation, preview, orientation toggle, and per-unit PDF export in DeclarationLabelPage. |

## 5.2 Business Rules

**Table 5.2.** Business Rules

| ID: Business Rule | Description |
|---|---|
| **Production Execution Rules** | |

Continued on next page

| ID: Business Rule | Description |
|---|---|
| BR-EXEC-001: Single Running Operation per Machine | A machine may only have one operation in the *Running* state at any given time. Before starting or resuming an operation, `EnsureNoRunningOperation()` queries all MES Operation Execution records for the machine and verifies that none of their latest MES Operation Status records has the status *Running*. A violation triggers an error with the conflicting order and operation numbers included in the message. |
| BR-EXEC-002: Declared Quantity Cannot Exceed Remaining Quantity | During `TryDeclareProduction()`, the sum of `TotalProducedQuantity` from the latest progression record and the new `input` is compared to `MESExecution."Order Quantity"`. If the sum exceeds the order quantity, an error is raised and no record is inserted. |
| BR-EXEC-003: State Machine for Operation Status | Operation status transitions are strictly enforced:<br>• *Running* → *Paused* (via `TryPauseOperation`).<br>• *Paused* → *Running* (via `TryResumeOperation`).<br>• *Running* or *Paused* → *Finished* or *Cancelled* (via `TryCloseOperation`).<br>• *Finished* or *Cancelled* → any other state (blocked: `TryCloseOperation` raises an error).<br>Each transition inserts a new immutable MES Operation Status record; no records are ever updated or deleted, providing a full audit trail. |
| BR-EXEC-004: Machine Status is Derived from Operation Status | MES Machine Status records are inserted (never updated) whenever an operation transition occurs:<br>• Start / Resume → insert *Working* status with the current production order number.<br>• Pause / Finish / Cancel → insert *Idle* status with no production order.<br>The machine list display reads the most recent record for each machine (sorted by Last Updated At descending). |
| BR-EXEC-005: Only Released Orders Can Be Started | `TryStartOperation()` filters Prod. Order Routing Line with `Status = Released`. Planned and Firm Planned lines are visible in the Flutter Orders tab but their Start Order button is disabled (`canStart = order.status == 'Released'`). |

Continued on next page

| ID: Business Rule | Description |
|---|---|
| **Scrap Declaration Rules** | |
| `BR-SCRAP-001`: Scrap Requires Active Operation | • Scrap can only be declared when the operation status is *Running*.<br>• The *Report Reject* button is disabled for *Paused*, *Finished*, and *Cancelled* states. |
| `BR-SCRAP-002`: Reason Code Mandatory | • A scrap reason code must be selected before submission.<br>• Codes are fetched from the standard Business Central Scrap Code table and cached in MesScrapProvider after the first load. |
| **Proxy Declaration Rules** | |
| `BR-PROXY-001`: Supervisor Attribution | • Only users with the Supervisor role see the operator selector in the declare production dialog.<br>• The selected operator's User Id is written to the `Declared By` field; the session User Id is written to `Operator Id` as the executing party.<br>• If no operator is selected, both fields contain the supervisor's User Id. |
| **Barcode Scanning Rules** | |
| `BR-SCAN-001`: Internal Barcode Priority | • If a scanned DataMatrix string matches the proprietary pipe-separated format, it is parsed locally without a server round-trip.<br>• External barcodes that do not match this format are resolved via `resolveBarcode()` using the `Item Identifier` table. |
| `BR-SCAN-002`: Duplicate Scan Merging | • Scanning the same item twice increments the quantity on the existing row in the pending scan list rather than adding a duplicate entry.<br>• The operator can adjust the quantity further using the +/− controls before confirming. |
| **Bill of Materials Rules** | |

Continued on next page

| ID: Business Rule | Description |
|---|---|
| BR-BOM-001: Routing Link Code Scoping | When an order's components have Routing Link Codes assigned, `fetchBom()` uses the following three-condition filter to determine which components to show for a given operation:<br>• **Include**: components whose Routing Link Code matches the current operation's code (`belongsToThisOperation = true`).<br>• **Include**: components with an empty Routing Link Code when at least one component in the order has a routing link (shown as shared, `belongsToThisOperation = false`).<br>• **Exclude**: components whose Routing Link Code is non-empty and differs from the current operation's code (they belong to a different operation).<br>When no component in the order has a Routing Link Code, all components are shown as operation-specific. |
| **Traceability Rules** | |
| BR-TRACE-001: Immutable Audit Records | MES Operation Status, MES Operation Progression, MES Machine Status, and MES Component Consumption records are insert-only. No update or delete operations are performed on these tables outside of testing pages, ensuring a complete chronological audit trail for every machine, order, and operator action. |
| BR-TRACE-002: Operator Identity on Every Record | All progression, status, and consumption records stamp the `Operator Id` using the BC session `UserId` at insert time. The `fetchProductionCycles()` endpoint resolves first and last name from the Employee table for display. |

## 5.3 Design

### 5.3.1 Use Case Diagram

This sprint adds production execution actors and interactions centred on the Operator and Supervisor roles. The primary use cases are: Declare Production, Pause Operation, Resume Operation, Finish Operation, Cancel Operation, View BOM Components, View Operation Detail, View Machine History, and Change Password.
The Supervisor role is a superset of Operator: both roles can declare production, pause, resume, and view detail; only the Supervisor may close (finish or cancel) a production order in the intended business flow, although the current implementation enforces this at the UI layer through the `canCloseProductionOrder` flag rather than at the API level. The resulting use case diagram is shown in Figure 5.1.

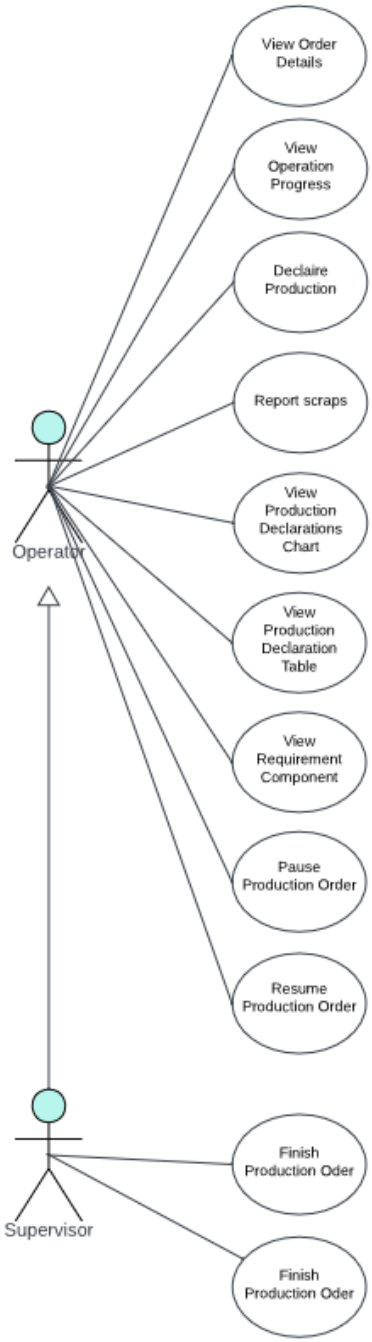


**Figure 5.1.** UML Use Case Diagram — Sprint 3

### 5.3.2 Textual Use Case Description

**Table 5.3.** Textual Use Case Description — Declare Production

| | |
|---|---|
| **Use Case Name** | Declare Production |
| **Primary Actor** | Operator / Supervisor |
| **Preconditions** | • The user is authenticated and navigated to an Operation Detail page.<br>• The operation status is *Running* or *Paused*.<br>• The produced quantity is below the order quantity. |
| **Postconditions** | • A new MES Operation Progression record is inserted.<br>• The production chart and cycle table update via the refresh stream.<br>• The progress percentage and produced quantity in the info container update. |
| **Main Scenario** | 1. The user taps *Declare Production* in the Quick Actions panel.<br>2. The DeclareProductionDialog opens showing the remaining quantity.<br>3. The user enters a positive integer not exceeding the remaining quantity.<br>4. The user taps *Submit*.<br>5. The system POSTs the declaration to `/MESWebService_declareProduction`.<br>6. The ERP validates the quantity and inserts the progression record.<br>7. The dialog closes and the refresh stream triggers a live data update. |
| **Alternative Flow** | • *Step 3, invalid quantity*: client-side validation shows an inline error; submission is blocked.<br>• *Step 6, ERP error*: the server error message is displayed in the dialog; the record is not inserted. |

**Table 5.4.** Textual Use Case Description — Finish / Cancel Operation

| | |
|---|---|
| **Use Case Name** | Finish / Cancel Operation |
| **Primary Actor** | Supervisor |
| **Preconditions** | • The user is authenticated with an active session.<br>• The operation is in *Running* or *Paused* state. |
| **Postconditions** | • A *Finished* or *Cancelled* status record is inserted.<br>• The execution end time is stamped.<br>• An Idle machine status record is inserted.<br>• The user is navigated back to the machine page. |
| **Main Scenario** | 1. The user taps *Finish Production Order* (if complete) or *Cancel Production Order* (if incomplete).<br>2. A confirmation dialog is displayed; the user confirms.<br>3. The system POSTs to `/MESWebService_finishOperation` or `cancelOperation`.<br>4. The ERP validates, inserts the status record, stamps the end time, and inserts an Idle machine status.<br>5. Flutter calls `Navigator.pop()` to return to the machine page. |

### 5.3.3 Sequence Diagrams

The following sequence diagrams illustrate the communication flow between the Flutter frontend, the Business Central OData layer, and the AL business logic for each machine and operation flow. Figures 5.2 through 5.5 present the four main interaction sequences.

Figure 5.2 depicts the production declaration flow, in which the operator submits a quantity, the application validates it locally, and Business Central either persists a new MES Operation Progression record or returns a validation error.

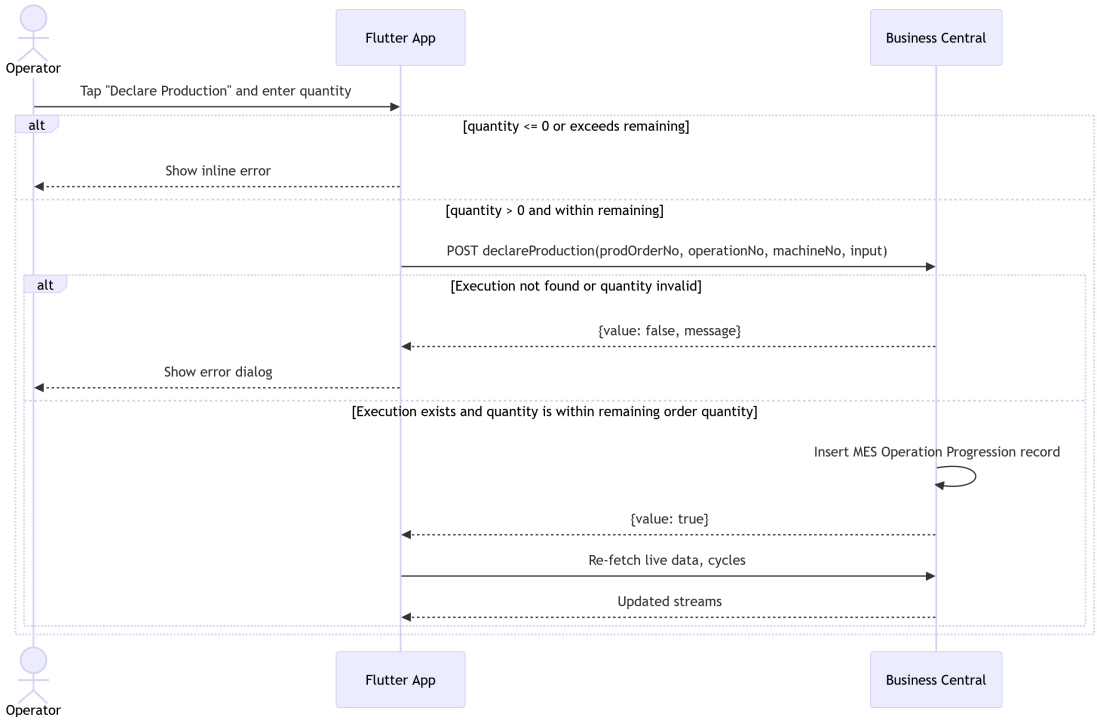


**Figure 5.2.** Sequence Diagram — Declare Production.

Figure 5.3 shows the BOM retrieval sequence, in which the application queries Business Central for the production order components scoped to the current routing operation, including planned, scanned, consumed, and remaining quantities per component.

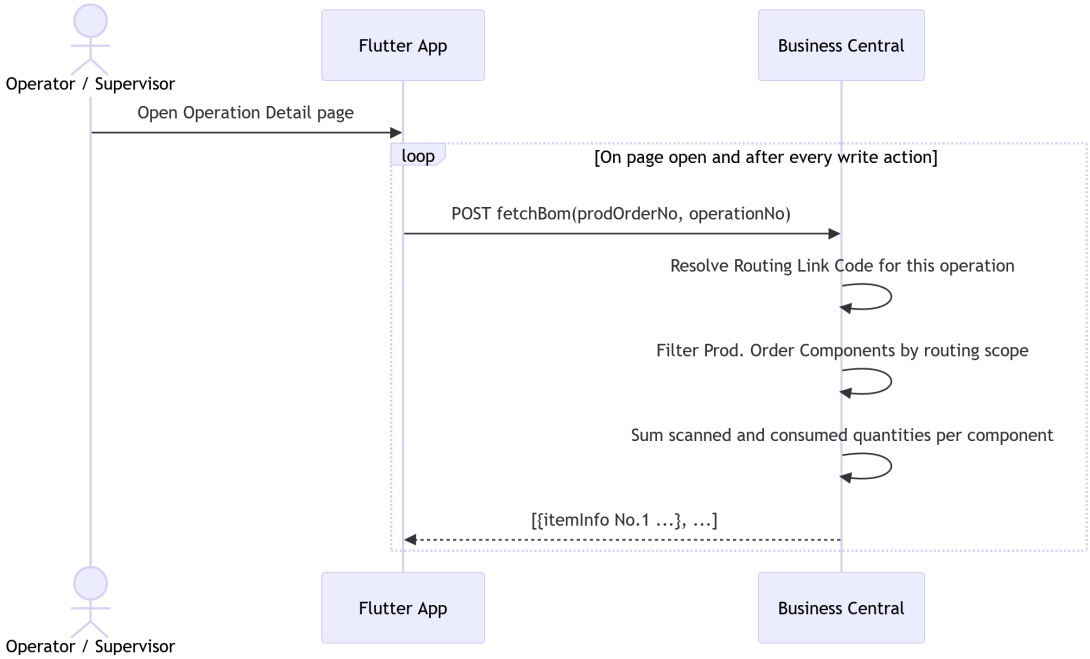


**Figure 5.3.** Sequence Diagram — Fetch Bill of Materials.

Figure 5.4 illustrates the production cycle retrieval sequence, in which the application fetches

MES Operation Progression records ordered from newest to oldest, with operator names resolved from the Employee table.

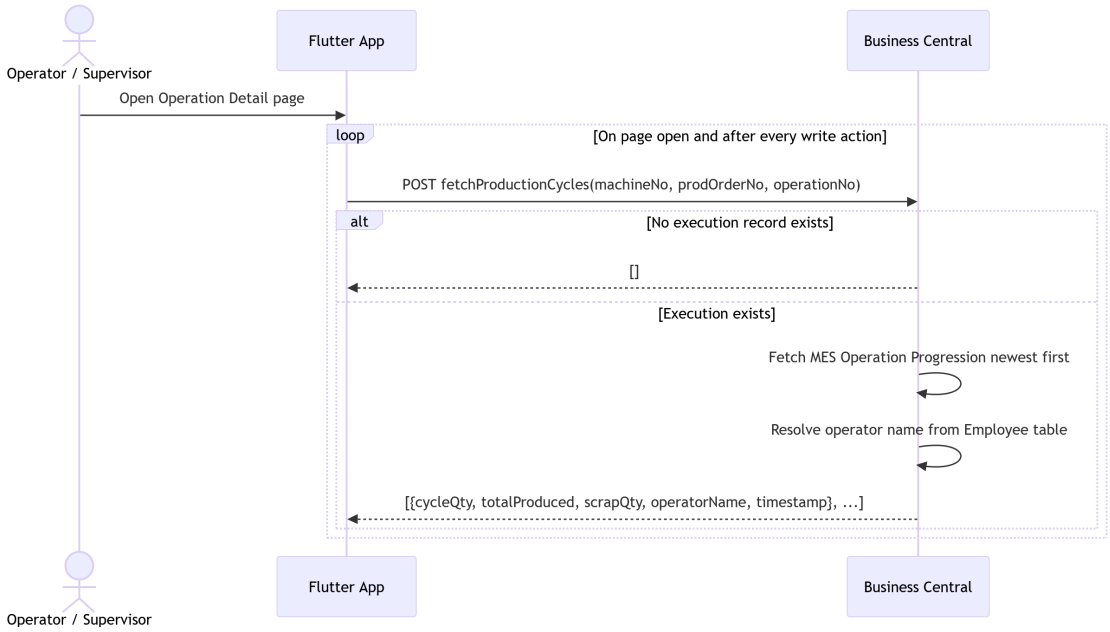


**Figure 5.4.** Sequence Diagram — Fetch Production Cycles.

Figure 5.5 presents the live operation data retrieval sequence, in which Business Central returns the current operation status, total produced quantity, scrap quantity, and progress percentage, or an empty response when the operation is finished, cancelled, or not yet started.

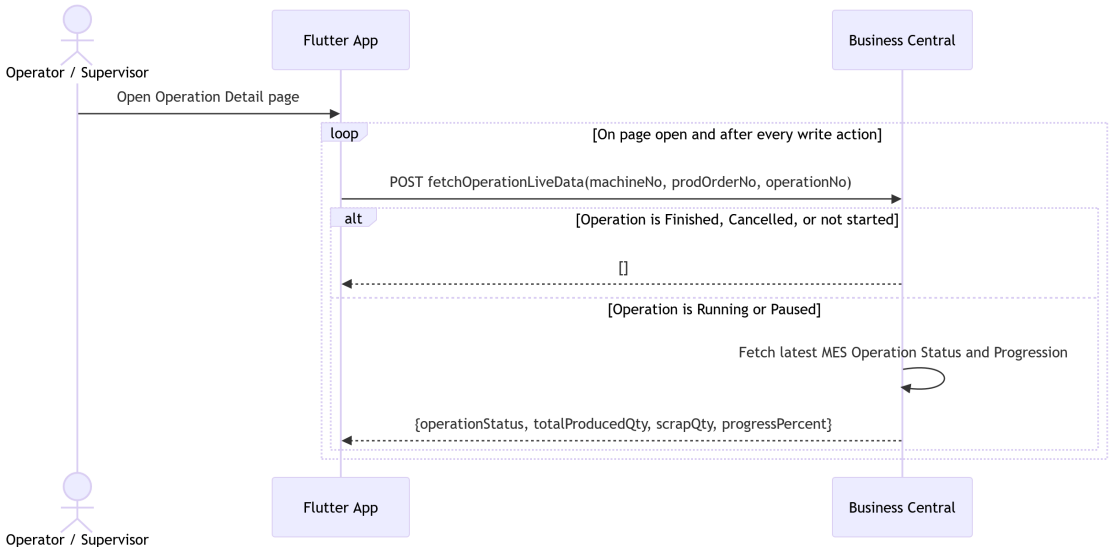


**Figure 5.5.** Sequence Diagram — Fetch Operation Live Data.

Figures 5.6 through 5.9 illustrate the remaining operation lifecycle transition flows: pause, resume, finish, and cancel. All four follow the same pattern as the start operation sequence — the

Flutter frontend POSTs to the corresponding MES Web Service endpoint, the AL backend validates the current state via the `Try*` family of functions, inserts a new immutable status record, and returns a structured success/error response.

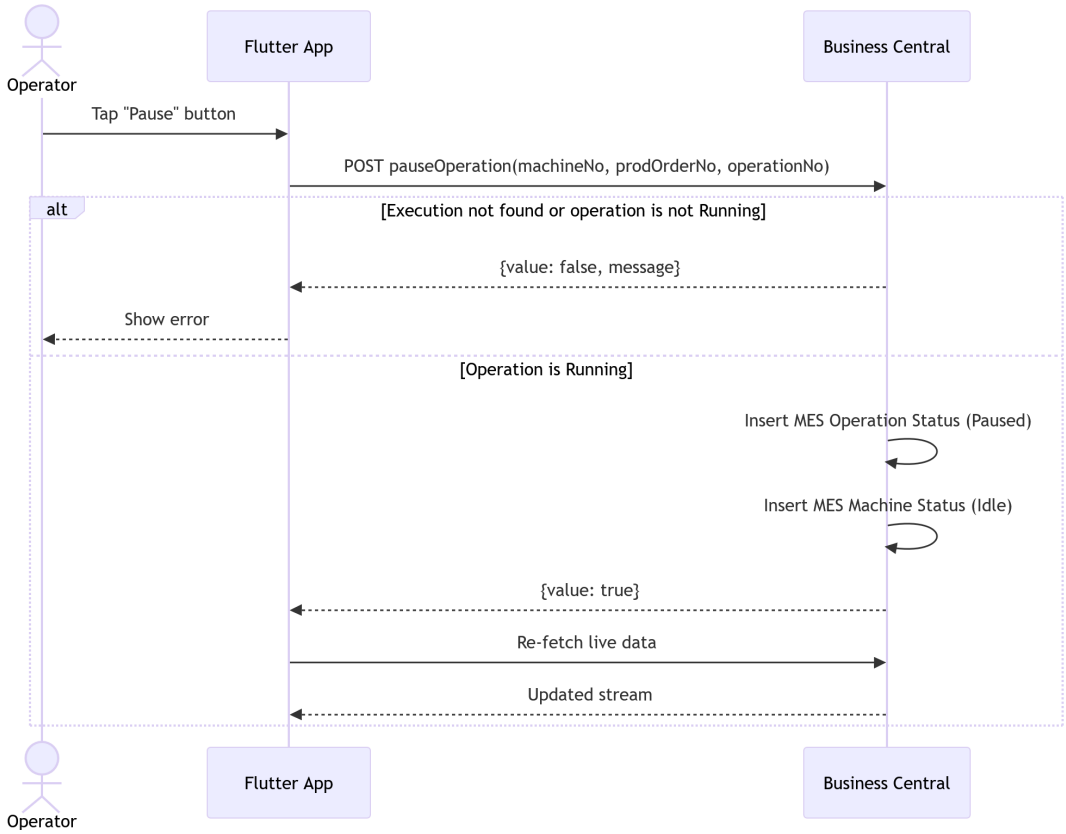


**Figure 5.6.** Sequence Diagram — Pause Operation.

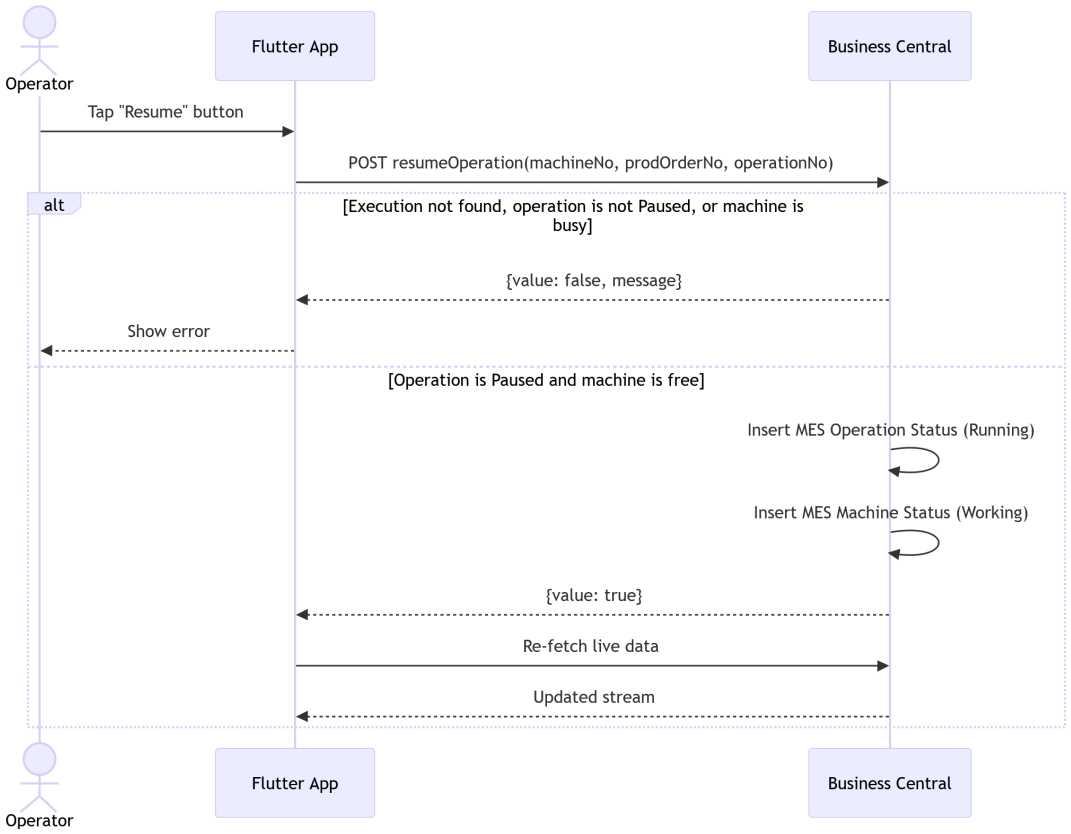


**Figure 5.7.** Sequence Diagram — Resume Operation.

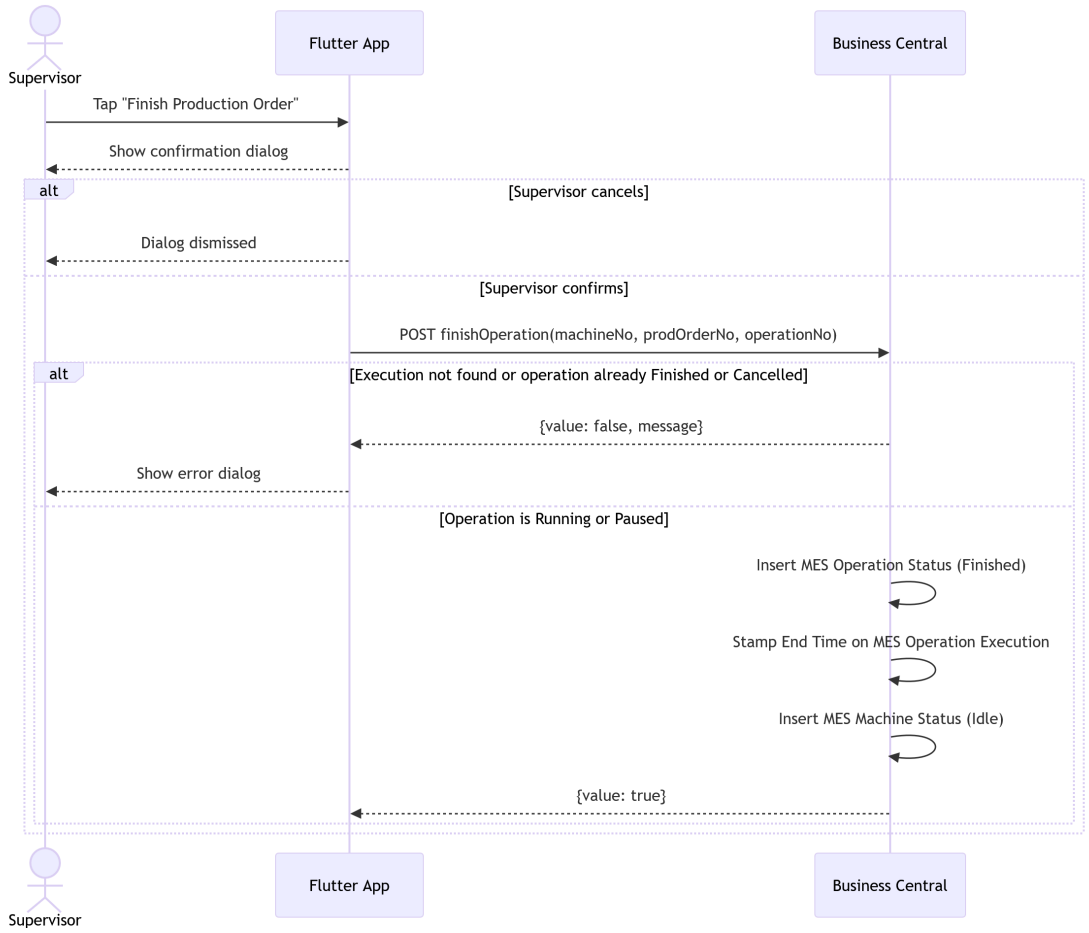


**Figure 5.8.** Sequence Diagram — Finish Operation.

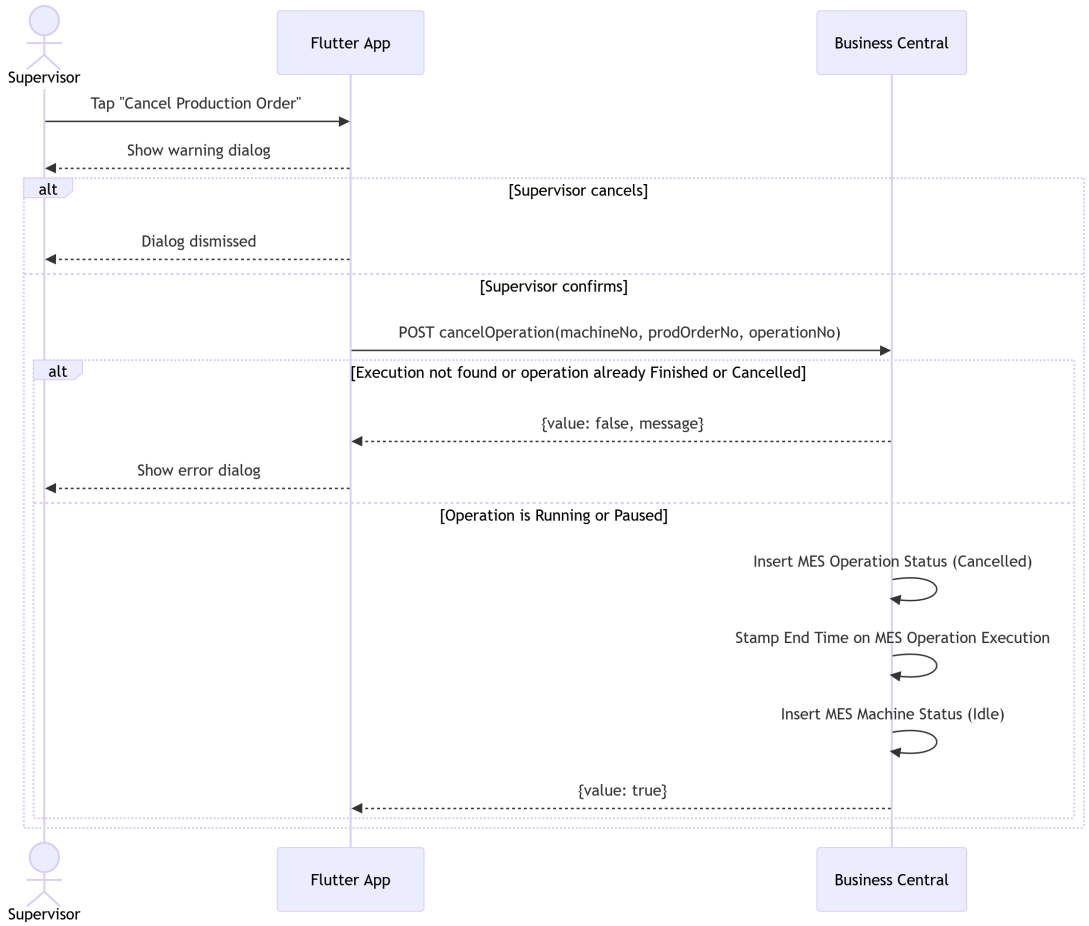


**Figure 5.9.** Sequence Diagram — Cancel Operation.

### 5.3.4 Class Diagram

The UML Class Diagram (Figure 5.10) extends the Sprint 2 domain model with three new tables. `MES Operation Execution` (50110) is the central anchor record linking a machine, production order, and operation to its full lifecycle; it holds item description, order quantity, and start/end timestamps. `MES Operation Progression` (50109) is an append-only table of production cycle declarations, each referencing an execution and accumulating `TotalProducedQuantity` from the previous cycle. `MES Component Consumption` (50111) records scrap and barcode scan events, also referencing an execution. Both progression and consumption records are child tables of the execution record, forming a hub-and-spoke traceability model.

**Figure 5.10.** UML Class Diagram — Production Execution & Traceability.

## 5.4 Implementation

### 5.4.1 Backend Implementation

#### Tables and Entities

**Table 5.5.** MES Operation Execution (Table 50110) — Field Dictionary

| Name | Type | Description |
|---|---|---|
| Execution Id | Code[50] | GUID-based primary key; auto-generated on insert. |
| Machine No | Code[20] | Foreign key to Machine Center. |
| Prod Order No | Code[20] | Foreign key to Prod. Order Routing Line. |
| Operation No | Code[10] | Operation identifier from the routing line. |
| Item No | Code[20] | Copied from Prod. Order Line on execution creation. |
| Item Description | Text[100] | Copied from Prod. Order Line for denormalized display. |
| Order Quantity | Decimal | Copied from Prod. Order Line; used for progress calculation. |

Continued on next page

| Name | Type | Description |
|---|---|---|
| Start Time | DateTime | Set by `OnInsert` trigger to `CurrentDateTime()`. |
| End Time | DateTime | Stamped when status transitions to Finished or Cancelled. |

**Table 5.6.** MES Operation Progression (Table 50109) — Field Dictionary

| Name | Type | Description |
|---|---|---|
| Id | Code[50] | GUID-based primary key; auto-generated on insert. |
| Execution Id | Code[50] | Foreign key to MES Operation Execution. |
| Cycle Quantity | Decimal | Units declared in this single production cycle. |
| Scrap Quantity | Decimal | Units scrapped in this cycle (currently always 0). |
| Total Produced Quantity | Decimal | Running total: previous total + Cycle Quantity. |
| Operator Id | Code[50] | BC session UserId of the operator who declared. |
| Last Updated At | DateTime | Set by `OnInsert` trigger to `CurrentDateTime()`. |

**Table 5.7.** MES Component Consumption (Table 50111) — Field Dictionary

| Name | Type | Description |
|---|---|---|
| Id | Code[50] | GUID-based primary key; auto-generated on insert. |
| Execution Id | Code[50] | Foreign key to MES Operation Execution. |
| Prod Order No | Code[50] | Foreign key to Prod. Order Component. |
| Item No | Code[50] | Item identifier of the consumed component. |
| Barcode | Code[50] | Barcode scanned to register the component. |
| Unit of Measure | Code[50] | Unit of measure code from the component definition. |
| Quantity Scanned | Decimal | Raw quantity read from the barcode scan. |
| Quantity Consumed | Decimal | Quantity actually applied to the production order. |
| Operator Id | Code[50] | BC session UserId of the operator who scanned. |
| Scanned At | DateTime | Set by `OnInsert` trigger to `CurrentDateTime()`. |

### REST API and Web Services Implementation

All endpoints are exposed as ODataV4 unbound actions through MES Web Service (codeunit 50126), reachable at:

```
POST <baseUrl>/ODataV4/MESWebService_<ProcedureName>?company=<companyId>
```

Table 5.8 lists all endpoints added in this sprint.

**Table 5.8.** New API Endpoints — Sprint 3

| Endpoint | Key Parameters | Response |
|---|---|---|
| `startOperation` | prodOrderNo, operationNo, machineNo | `{value: bool, message?}` |
| `declareProduction` | machineNo, prodOrderNo, operationNo, input | `{value: bool, message?}` |
| `pauseOperation` | machineNo, prodOrderNo, operationNo | `{value: bool, message?}` |
| `resumeOperation` | machineNo, prodOrderNo, operationNo | `{value: bool, message?}` |
| `finishOperation` | machineNo, prodOrderNo, operationNo | `{value: bool, message?}` |
| `cancelOperation` | machineNo, prodOrderNo, operationNo | `{value: bool, message?}` |
| `fetchOperationsStatusAndProgress` | machineNo, fetchFinished | JSON array of operation progress objects |
| `fetchOperationLiveData` | machineNo, prodOrderNo, operationNo | JSON array (0 or 1 element) |
| `fetchProductionCycles` | machineNo, prodOrderNo, operationNo | JSON array of cycle records |
| `fetchBom` | prodOrderNo, operationNo | JSON array of BOM component records |
| `ChangePassword` | token, oldPassword, newPassword | `{success: bool, message}` |

### 5.4.2 Frontend Implementation

This sprint introduces a key reactive pattern: a shared `StreamController<void>.broadcast()` in MachineordersProvider and MesComponentConsumptionProvider. When any write action (declare, pause, resume, finish, cancel) succeeds, `triggerRefresh()` adds an event to the controller. All live data streams (`fetchOperationLiveDataStream`, `streamProductionCycles`, `streamBom`) are implemented as `async*` generators that yield on both the initial fetch and on each event from the shared broadcast stream, enabling coordinated, push-driven UI updates without polling.

**Provider Architecture**

**Table 5.9.** Provider Responsibilities — Sprint 3 Additions

| Provider | Responsibility |
|---|---|
| MachineordersProvider | Owns write operations (start, declare, pause, resume, finish, cancel) and their refresh controller; exposes all live data streams. |
| MesComponentConsumption Provider | Exposes `getBomStream()` combining service fetch with an independent refresh controller. |
| AuthProvider | Adds `changePassword()` and `adminSetPassword()`; manages `needsPasswordChange` flag in the in-memory session. |

**User Interface**

Figures 5.11 through 5.14 illustrate the *Operation Detail Page* across its four possible states: Running, Paused, Finished, and Cancelled. Each view adapts its action buttons and status badge to the current state, while the production chart, cycle table, and BOM list remain consistently visible. On desktop, the page uses a two-column layout with order information and production data on the left, and the *Quick Actions* panel on the right; on mobile, these sections stack vertically. Action button availability is state-dependent: *Declare Production* and *Report Reject* are disabled in the Paused, Finished, and Cancelled states; the close button triggers a *Finish Production Order* or *Cancel Production Order* action depending on whether the progress has reached 100%; and *Print Label* is enabled only once the operation is finished.

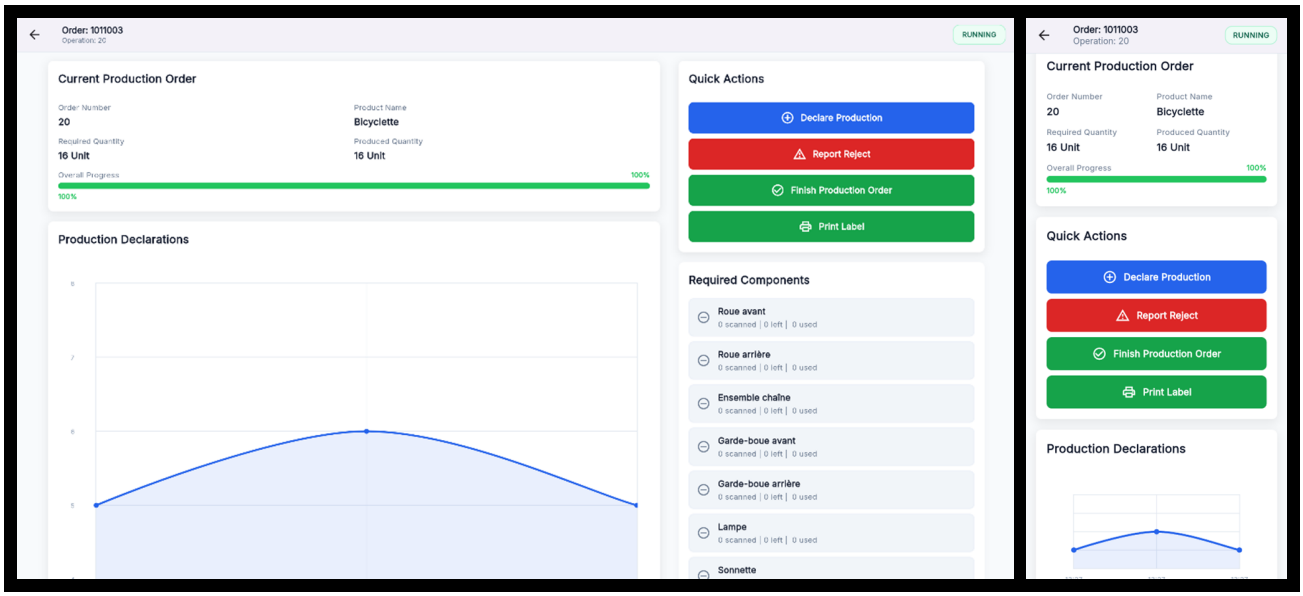


**Figure 5.11.** *Operation Detail Page* (Running state) — desktop and mobile views.

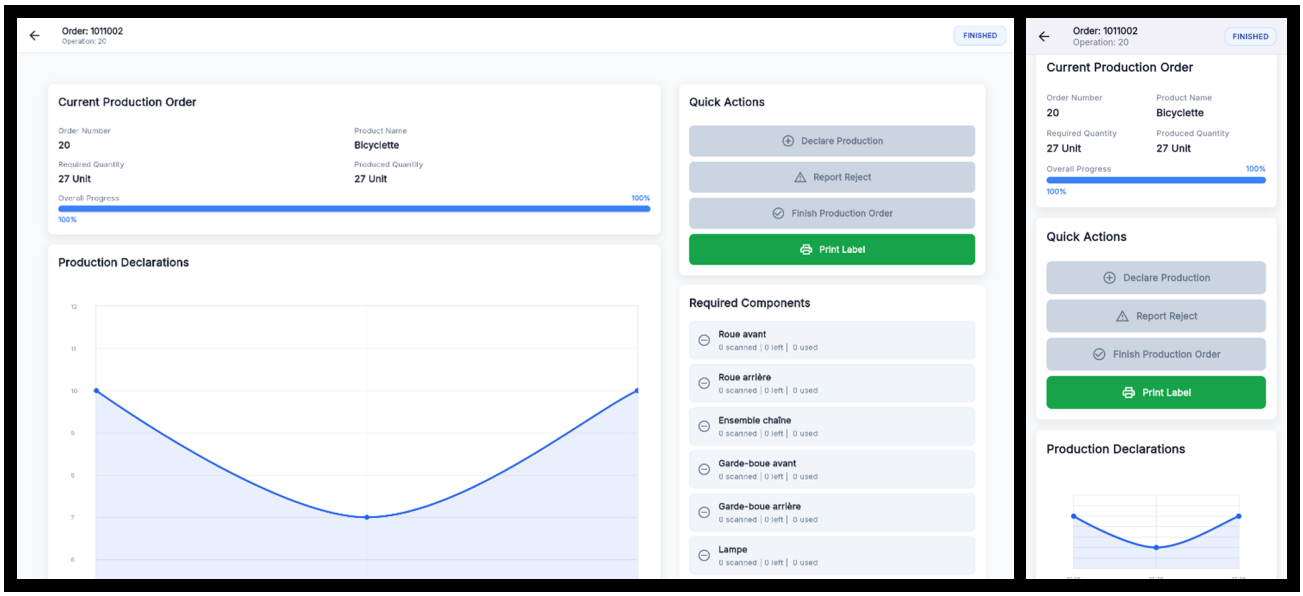


**Figure 5.12.** *Operation Detail Page* (Finished state) — desktop and mobile views.

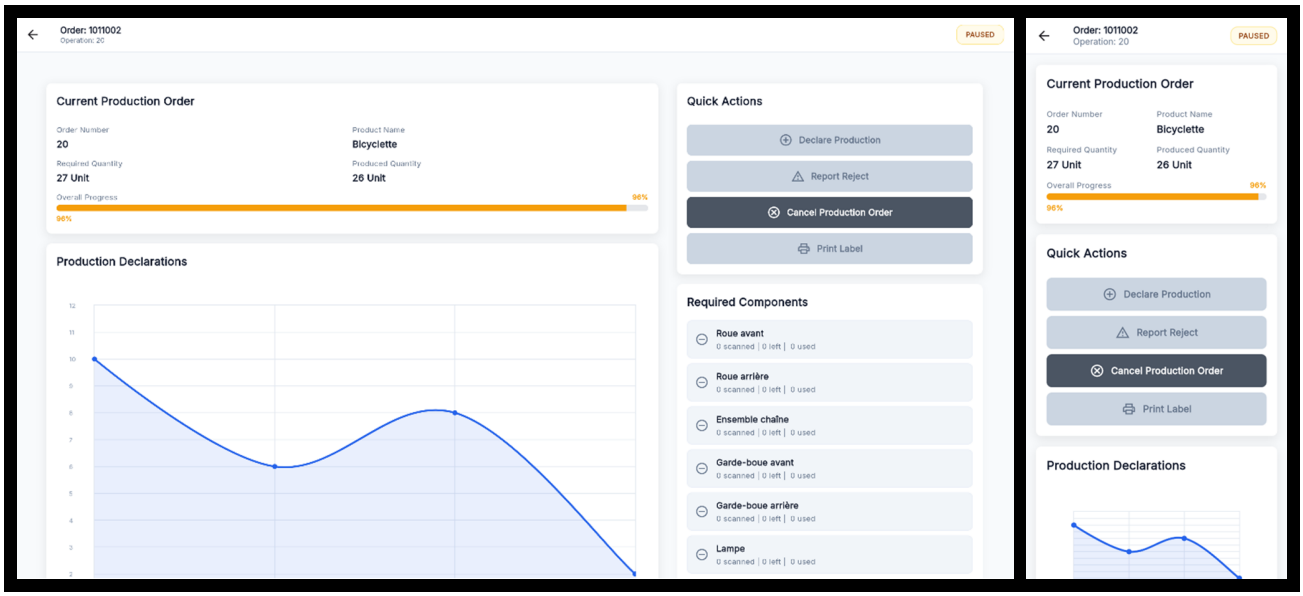


**Figure 5.13.** *Operation Detail Page* (Paused state) — desktop and mobile views.

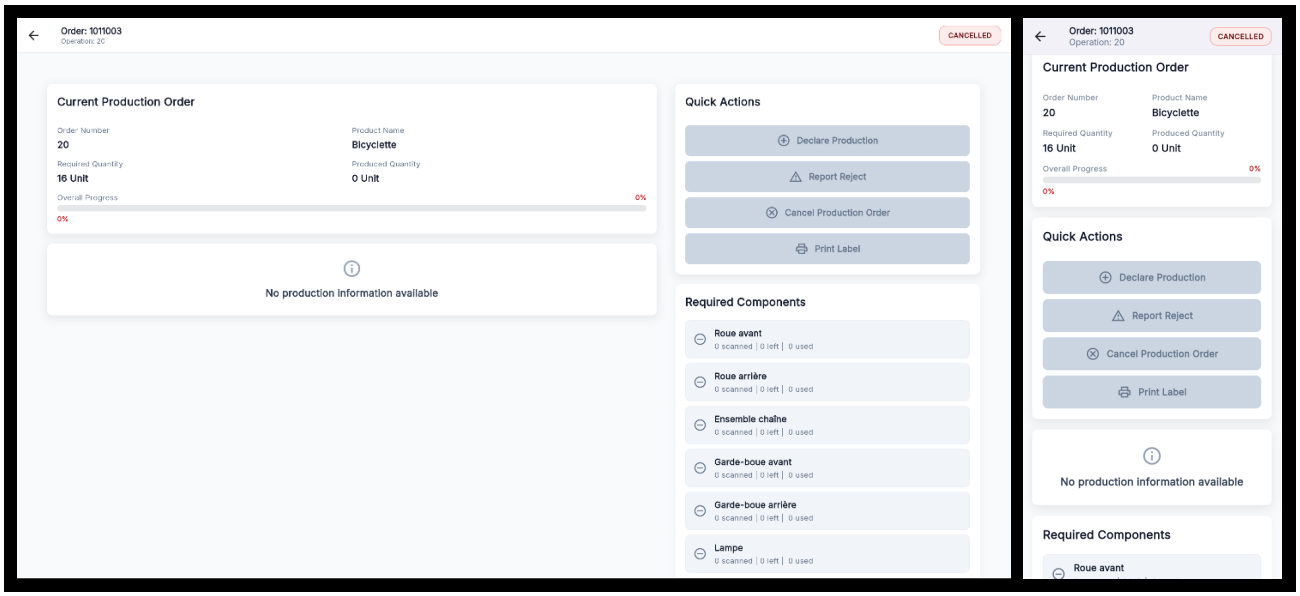


**Figure 5.14.** *Operation Detail Page* (Cancelled state) — desktop and mobile views.

## Conclusion

This sprint delivered the complete shop-floor execution loop for the MES system. Operators can now start, pause, resume, and close production operations, declare production output cycle by cycle, track cumulative progress through real-time charts and paginated tables, inspect the Bill of Materials, scan physical components via DataMatrix barcodes, declare scrapped units with reason codes, and print traceable labels for both production orders and individual declared units. Supervisors gain the ability to declare production on behalf of a specific operator, with a full attribution audit trail. The reactive stream architecture ensures that all UI panels update

in concert after every write action, providing a consistent and accurate view of the production floor without manual refreshes.

# Chapter 6

# MES Sprint 4 Report

## Contents

## Introduction

This sprint delivers the administration layer, multi-language support, and in-app onboarding tutorials that complete the *MES* solution. Administrators receive a persistent navigation shell giving access to a machine performance dashboard, a paginated activity log, and a user role management interface. Cross-cutting quality-of-life features include full internationalisation with English, French, and Arabic support, automatic right-to-left layout for Arabic, and guided coach-mark tutorials shown once per screen on first use. No new Business Central tables are introduced in this sprint; all data is derived by querying and aggregating the tables created in Sprints 1 through 3.

## 6.1 Sprint Backlog

This sprint covers **Domain 4:** ***Administration & Monitoring*** and **Domain 5:** ***UX & Accessibility***.

**Table 6.1.** sprint-backlog

| ID | User Story | Tasks |
|---|---|---|
| **US-4.1** | As an Administrator, I need a performance dashboard for all machines so that I can monitor production health across the facility. | *Business Central (AL):*<br>• Created function `fetchMachineDashboard()` in codeunit MES Machine Fetch.<br>• exposed the function through the codeunit MES Web Service.<br>*Flutter:*<br>• Created model MachineDashboardModel.<br>• Created LogService to consume the `fetchMachineDashboard` API and LogProvider to manage its state.<br>• Created page MachineDashboardPage.<br>• Created widget AdminMachineCard.<br>• Implemented machine dashboard search and time-range filtering in MachineDashboardPage. |
| **US-4.2** | As an Administrator, I need a filterable, paginated log of all system actions so that I can audit operator and machine activity. | *Business Central (AL):*<br>• Created function `fetchActivityLog()` in codeunit MES Machine Fetch.<br>• added the wrapper of the function to MES Web Service. |

Continued on next page

| ID | User Story | Tasks |
|---|---|---|
| | | *Flutter:*<br>• Created model ActivityLogModel.<br>• Created LogService to consume the `fetchActivityLog()` API and LogProvider to manage its state.<br>• Created page ActivityLogPage.<br>• Implemented activity log search, type filtering, time-range filtering, and pagination in ActivityLogPage. |
| **US-4.3** | As an Administrator, I need a persistent sidebar so that I can navigate between all admin sections without losing context. | *Flutter:*<br>• Created page AdminPage.<br>• Created widget SidebarItem.<br>• Implemented sidebar navigation and profile-page navigation in AdminPage. |
| **US-4.4** | As any user, I need the application available in English, French, and Arabic so that I can use it in my preferred language. | *Flutter:*<br>• Integrated the `easy_localization` package.<br>• Created translation JSON files for `en`, `fr`, and `ar`.<br>• Created widget LanguageSelector.<br>• Implemented language selection in ProfilePage.<br>• Added the EasyLocalization wrapper in `main()`.<br>• Implemented right-to-left layout support through `MaterialApp`. |
| **US-4.5** | As a first-time user, I need guided coach-mark tutorials so that I can learn the interface without external documentation. | *Flutter:*<br>• Integrated the `tutorial_coach_mark` package.<br>• Implemented tutorial persistence using SharedPreferences.<br>• Created MachineListTutorial.<br>• Created MachineDetailTabsTutorial.<br>• Created OperationDetailTutorial.<br>• Created AdminDashboardTutorial.<br>• Implemented tutorial target styling and localisation. |

## 6.2 Business Rules

**Table 6.2.** Business Rules

| ID: Business Rule | Description |
|---|---|
| **Dashboard and Log Rules** | |
| BR-ADMIN-001: Dashboard Time-Range Scoping | • The hoursBack parameter filters all dashboard and activity log queries; it is applied server-side when querying the MES tables.<br>• The default value is 24 hours; available options are exposed as hourOptions in LogProvider (e.g., 8 h, 24 h, 48 h, 7 days).<br>• Changing the time-range selector triggers a new API call and replaces the displayed data. |
| **Internationalisation Rules** | |
| BR-I18N-001: RTL Support | • Selecting the Arabic locale (ar) sets the MaterialApp locale to ar; Flutter's Directionality widget handles right-to-left layout automatically without any per-widget configuration.<br>• All UI strings are externalised in JSON translation files; no hard-coded string literals remain in the codebase.<br>• The language preference is persisted between app restarts via the EasyLocalization storage layer. |
| **Tutorial Rules** | |
| BR-TUTORIAL-001: One-Time Display | • Each tutorial is shown exactly once per application install, gated by a SharedPreferences boolean flag keyed to the tutorial identifier (e.g., tutorial_machine_list_shown).<br>• Once dismissed or completed, the flag is set to true and the tutorial is never shown again, even after the application is restarted. |

## 6.3 Design

### 6.3.1 Use Case Diagram

The UML Use Case Diagram (Figure 6.1) covers the actors and interactions introduced in this sprint. The Administrator gains three new interactions: *View Machine Dashboard*, *View Activity Log*, and *Navigate Admin Shell*. All three actors (Operator, Supervisor, Administrator) share the *Select Language* and *View Tutorial* use cases, which are cross-cutting accessibility features

available on every screen.



**Figure 6.1.** UML Use Case Diagram — Administration & UX.

### 6.3.2 Textual Use Case Description

The following table (Table 6.3) provides the detailed textual description of the *View Machine Dashboard* use case, which represents the most significant new administrator workflow in this sprint.

**Table 6.3.** Textual Use Case Description — View Machine Dashboard

| | |
|---|---|
| **Use Case Name** | View Machine Dashboard |
| **Primary Actor** | Administrator |
| **Preconditions** | • The user is authenticated with the *Admin* role.<br>• At least one MES Operation Execution record exists. |
| **Postconditions** | • A grid of machine cards is displayed, each showing uptime, production totals, and scrap for the selected time window.<br>• Changing the time-range selector refreshes all card metrics. |

Continued on next page

| | |
|---|---|
| **Main Scenario** | 1. The administrator logs in and is routed to AdminPage.<br>2. The administrator selects *Machine Dashboard* in the sidebar.<br>3. The system POSTs to `/MESWebService_fetchMachineDashboard` with the default `hoursBack = 24`.<br>4. The backend aggregates execution and progression records and returns per-machine metrics.<br>5. The MachineDashboardPage renders one AdminMachineCard per machine.<br>6. The administrator changes the time range to 48 h via the AppBar dropdown; the page re-fetches and updates. |

### 6.3.3 Sequence Diagrams

Figures 6.2 through 6.3 illustrate the two main backend query flows introduced in this sprint. Both follow the same pattern: the Flutter frontend sends a POST request with an admin session token and a `hoursBack` parameter; the Business Central backend validates the token, aggregates data from the Sprint 1–3 tables, and returns a JSON array.



**Figure 6.2.** Sequence Diagram — Fetch Machine Dashboard.

**Figure 6.3.** Sequence Diagram — Fetch Activity Log.

### 6.3.4 Class Diagram

The UML Class Diagram (Figure 6.4) shows the four Flutter-side models introduced in this sprint. `MachineDashboardModel` and `ActivityLogModel` are read-only projection models derived from the Sprint 1–3 tables; they have no corresponding AL table and are populated entirely from aggregated API responses. `MesScrapCode` is a lightweight reference model populated from the standard Business Central Scrap Code table. `ItemBarcodeModel` represents a scanned component item and includes a `toJson()` serialiser used when submitting scan batches to the `insertScans` endpoint.

**Figure 6.4.** UML Class Diagram — Administration & Scanning Models.

## 6.4 Implementation

### 6.4.1 Backend Implementation

No new Business Central tables are created in this sprint. All backend work consists of new query and aggregation procedures added to the existing MES Web Service codeunit (50126), reading from the tables defined in Sprints 1 through 3.

#### Dashboard Aggregation

`fetchMachineDashboard()` accepts a validated admin token and a `hoursBack` integer. It iterates all `Machine Center` records, and for each machine it filters `MES Operation Execution` records whose `Start Time` falls within the requested window. For each matched execution it sums the `Cycle Quantity` fields from linked `MES Operation Progression` records to obtain `totalProduced`, and counts records marked as scrap to obtain `totalScrap`. Running minutes are derived from the difference between `End Time` (or the current datetime for open executions) and `Start Time`. Uptime percentage is computed as `runningMinutes / (hoursBack × 60) × 100`, capped at 100.

#### Activity Log Aggregation

`fetchActivityLog()` queries three tables in descending timestamp order, filtered to the `hoursBack` window: `MES Operation Status` (type `status_change`), `MES Operation Progression` (type `production`), and `MES Component Consumption` (type `scan` or `scrap`

depending on whether a scrap code is present). For each record the operator's first and last name are resolved from the `Employee` table via the `Operator Id` foreign key. The results are merged into a single chronological list before being returned as a JSON array.

**API Endpoints Added in Sprint 4**

**Table 6.4.** New API Endpoints — Sprint 4

| Endpoint | Key Parameters | Response |
|---|---|---|
| `fetchMachineDashboard` | token, hoursBack | JSON array of `MachineDashboardModel` objects |
| `fetchActivityLog` | token, hoursBack | JSON array of `ActivityLogModel` objects |

### 6.4.2 Frontend Implementation

**Shared HTTP Infrastructure**

All services in this sprint (and retroactively migrated services from earlier sprints) use a shared HttpClient and HttpResponseParser layer, replacing the inline `http.post` calls used in Sprint 1.

`HttpClient.post()`: centralised `http.post` wrapper that injects JSON content-type headers and encodes the request body. All services call this method rather than the raw `http` package directly.

`HttpResponseParser`: provides four static methods. `parseList()` decodes Business Central OData list responses (nested value string). `parseObject()` decodes single-object responses. `parseWriteResult()` decodes write-result responses and throws with the backend error message on failure. `_assertSuccess()` validates HTTP status codes and throws a structured exception for non-2xx responses.

**Provider Architecture**

**Table 6.5.** Provider Responsibilities — Sprint 4 Additions

| Provider | Responsibility |
|---|---|
| LogProvider | Owns dashboard and activity log fetch calls; exposes `selectedHours`, `hourOptions`, and `setHours()` shared between the two admin pages. |

Continued on next page

| Provider | Responsibility |
|---|---|
| MesBarcodeProvider | Exposes `fetchAllBarcodes()`, `insertScans()`, and `resolveBarcode()`; resolves session token automatically from AuthProvider. |

### User Interface

***Admin Navigation Shell*** **(Figure 6.5)** provides the persistent sidebar that frames all administrator pages. A 250 px dark panel on the left lists the four sections (Users & Roles, Machine Dashboard, Activity Logs, Settings), with the active section highlighted in blue. At the bottom, a user-info strip shows the administrator's avatar, full name, and Auth ID, and navigates to ProfilePage on tap. The `IndexedStack` to the right keeps all pages alive simultaneously, preserving scroll position and loaded data as the administrator switches sections.



**Figure 6.5.** *Admin Navigation Shell* — desktop view.

Figures 6.6 and 6.7 illustrate the two main administration pages. The *Machine Dashboard Page* renders one AdminMachineCard per machine, each displaying a circular uptime indicator and four metric rows (operations, produced, scrap, running minutes); the AppBar hosts a time-range dropdown that re-fetches data on change. The *Activity Log Page* presents a paginated list of all system events ordered newest-first; each row shows a type icon, operator name, machine,

action text (expandable for long entries), and timestamp; type and time-range dropdowns in the AppBar narrow the view.

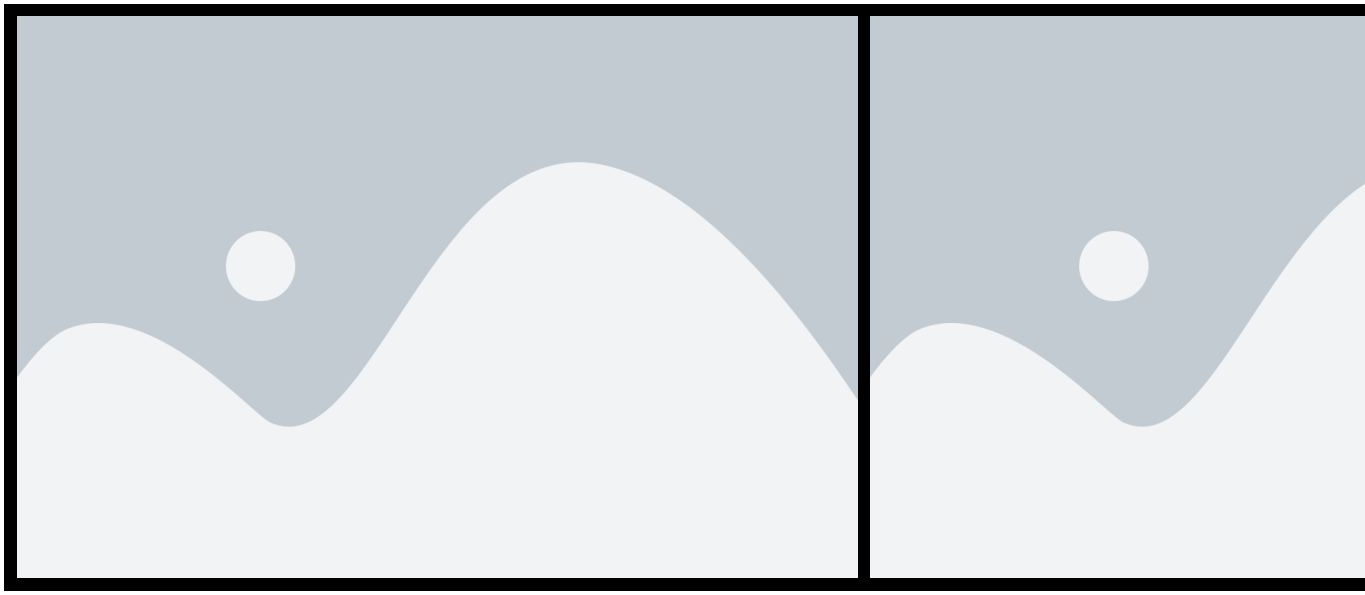

**Figure 6.6.** *Machine Dashboard Page* — desktop and mobile views.

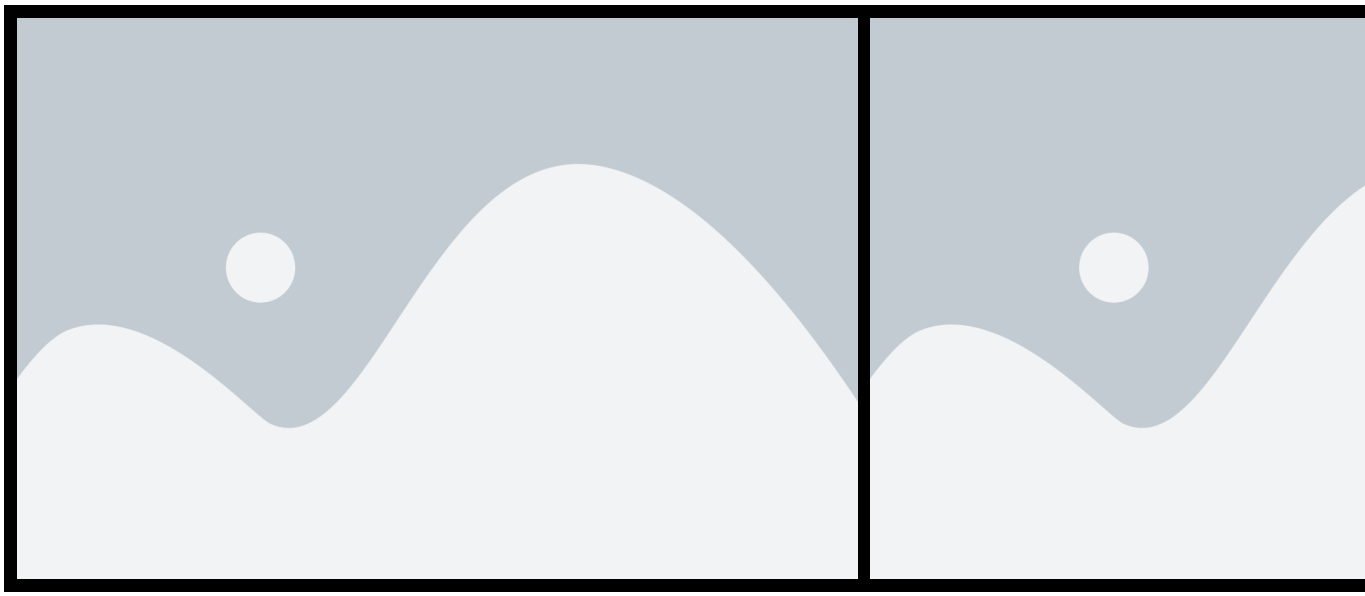

**Figure 6.7.** *Activity Log Page* — desktop and mobile views.

## Conclusion

This sprint completed the *MES* solution by delivering the administration layer and cross-cutting quality-of-life features. Administrators can now monitor the production health of the entire facility through a configurable machine performance dashboard, audit all operator and machine actions through a paginated activity log, and manage users from a persistent navigation shell without losing context between sections. All users benefit from full internationalisation cov-

ering English, French, and Arabic with automatic right-to-left layout, and from guided coachmark tutorials that ease onboarding for first-time users.

All business rules defined at the start of the sprint have been enforced: time-range scoping (`BR-ADMIN-001`), right-to-left layout and string externalisation (`BR-I18N-001`), and one-time tutorial display (`BR-TUTORIAL-001`). No new Business Central tables were required; all data is derived from the tables created in Sprints 1 through 3, demonstrating the extensibility of the append-only architecture established from the first sprint.

# General Conclusion

This report has presented the full lifecycle of the MES (*Manufacturing Execution System*) project developed at **Mavision** over the course of four sprints. Starting from a blank slate, the team designed and implemented a complete shop-floor execution system natively integrated with Microsoft Dynamics 365 Business Central.

Sprint 1 established the security and identity foundation: a token-based authentication system, role-based access control, and a full administrator interface for user and account management. Sprint 2 delivered real-time machine monitoring, production order management, and a complete operation history trail. Sprint 3 completed the production execution loop with cycle declarations, pause, resume, and close flows, Bill of Materials visibility, component scanning, scrap tracking, and label printing. Sprint 4 added the administration layer with a machine performance dashboard, a paginated activity log, multi-language support covering English, French, and Arabic, and in-app onboarding tutorials.

Across all four sprints, the append-only data model ensures a tamper-evident audit trail from every machine state change through to every production cycle declaration. The reactive stream architecture in the Flutter frontend provides operators and supervisors with a consistent, real-time view of the production floor without manual refreshes.

The project demonstrates that a tightly integrated MES solution can be built on top of Microsoft Dynamics 365 Business Central using OData V4 unbound actions, without requiring additional third-party middleware. The extensible layered architecture positions the system for future enhancements such as statistical reporting, alert management, and additional language support.

# Bibliography

[1] Mavision, *Official Website*, https://www.mavision.tn, visited April 2025.